

# 1 Introduction

## 1.1 Reproducibility is not claim integrity

A reproducible artifact is one that regenerates its own results. That is a real and necessary property. It is also, on the evidence we report here, a much weaker property than the one a reader of a paper actually relies on. A reader wants to know that the number in Table I is the number the code produces, that the method described in Section III is the method the artifact implements, and that the conclusion in the abstract follows from the result in the body. Those are three different questions. Only the first is a reproducibility question.

The gap between the two is not a matter of degree. In our corpus, ten of thirteen artifacts reproduce under auditor execution, and five of the thirteen scored rows reproduce bit-for-bit, with a sixth doing so on every scientific field but one wall-clock field. Not one of the thirteen packages is fully supported. The reproduced group and the fully-supported group are disjoint, and we should be precise about what establishes that, because an earlier version of this paragraph called it machine-checked and it was not. The two sets are disjoint because the `SUPPORTED` column is empty; two disjoint sets of which one is empty are trivially disjoint, and an assertion that no row both reproduces and carries a `SUPPORTED` verdict is fully subsumed by the assertion that no row carries a `SUPPORTED` verdict. The check that carries content, and can fail, is the joint distribution: of the ten reproducing rows seven are `PARTIALLY SUPPORTED` and three are `CONTRADICTED`, and of the three that do not reproduce one is `PARTIALLY SUPPORTED` and two are `CONTRADICTED`. Reproduction is not a usable proxy for integrity, and it is not even weakly predictive of the verdict. `figures/make_figures.py` recomputes that joint distribution on every run and aborts if a single reproduced row comes back `SUPPORTED`.

This is not a story about fabrication. The most reproducible artifact in our corpus belongs to a package whose central claim is refuted by a robustness file the package itself contains and never cites. Good simulation hygiene buys nothing when the defect lives in what the simulation measures.

## 1.2 Two principles

Every defect reported in this paper, in the thirteen audited packages and in our own apparatus alike, is an instance of one of two statements. We state them here because they are the synthesis of the series rather than a summary of it, and because a reader who holds them can classify a new defect without our taxonomy. Section 4.5 develops them against our own five non-arithmetic defects, which is where they earn the status.

*An integrity check cannot establish correspondence if its inputs are themselves an unaudited transcription of the claims being checked.*

*Verification should be derived from evidence, not from a transcription of the claim under verification.*

The first principle is about the *provenance* of a check’s inputs. A check that compares a transcribed copy of a number against a transcribed copy of the same number, both made by the same person from the same source, establishes that the transcriber was consistent with himself. It is not nothing, and in this series it caught a real row-sum error. It is not correspondence, and no amount of additional care by the same author can convert it into correspondence, because the property that would make it correspondence — an independent origin for the expected value — is exactly the property it lacks.

The second principle is about the *direction* of derivation, and it is the more general of the two. Where a check’s expected value is transcribed from the document that states the claim, the check is pointed at the layer it came from, and a defect in that layer is by construction outside its reach. The first principle is a special case: an unaudited transcription of the claims being checked is one way of deriving the expected value from the claim under verification. We state both because they name two different remedies — give the check an independently derived input, or derive the expectation from evidence — and because a reader repairing a broken apparatus needs to know which of the two is missing before choosing.

Both are stated as findings rather than as advice. Neither was known to us before the series ran: each is an account of why a specific instrument of ours was weaker than its own documentation claimed, and Section 6.4 gives the evidence for each.

## 1.3 The artifact-to-claim gap

The problem this paper addresses has a short name: *when running the code is not enough*.

Three failure modes recur, and they are not reducible to one another because they attack different links between a computational artifact and a scientific assertion. In the first, the artifact reproduces a property derived from the very assumption it is being tested against, so the validation is circular by construction. In the second, a reference implementation supplied as independent verification compiles, runs, exits cleanly, and returns a result that contradicts the paper it is bundled with, while the package’s own README instructs the reader to run it as a check. In the third, the artifact exists, produces results, and contains data structures named after the mechanism the paper describes, but the implemented mechanism is a different one; the naming is intact and the referent is not.

Every one of the three survives a clean reproduction and a careful number check. That is the methodological claim of this paper. A conventional reproducibility benchmark, and a conventional claim-level number check, both pass all three.

## 1.4 Research questions

The study asks four questions, stated so that each can fail.

- **RQ1.** Is artifact-to-claim correspondence materially worse than artifact reproducibility, under a rubric with a fixed unit of analysis?
- **RQ2.** Which failure classes of the artifact-to-claim relation does the gap decompose into, and at which unit of analysis is each one detectable?
- **RQ3.** Do reproduction and package-level support coincide, or are they independent properties in a corpus of real packages?
- **RQ4.** What survives? An audit that reports only negative findings is not a balanced audit, and the surviving results are what make the negative ones defensible.

## 1.5 Contributions

Our contributions are four. First, a four-unit analytic model — claim, artifact, relation, package — in which the relation is the unit the earlier layer rubric, v1.0 of our own, lacked, and a demonstration that the two failure classes living in the relation are exactly the ones a reproduction check cannot see. Second, a frozen, versioned rubric that fixes the unit of analysis before adjudication begins, and a correction log that records seven defects in our own aggregation layer and verification apparatus across five entries: three arithmetic, one a layer-label shift in which no count was wrong, one a rate attached to the wrong denominator span, and two in the apparatus, one of which is a provenance guarantee this paper published and did not implement. Three surfaced only after the freeze, and two only under an independent falsification attempt that imported no code of ours. The four non-arithmetic defects are then shown to be one phenomenon observed five times — once per carrier — rather than five unrelated mistakes, and the two principles of Section 1.2 are stated as its general form. The falsification attempt is itself part of the contribution, because it is the direct evidence for the limitation we now state as a finding: an apparatus whose checked source is a constant transcribed by its own author cannot be relied upon to detect a correspondence failure in itself. We report how we responded to that finding as a case of its own (Section 7.1): the claim was left standing and the evidence chain was raised until the claim became true. Third, a frozen dataset of thirteen scored rows over twelve distinct packages, with a per-row audit trail and a claim-granularity limitation reported rather than buried. Fourth, a taxonomy of three failure classes with worked cases, including a near-positive case that answers the obvious objection that a corpus of failures is not evidence of anything.

## 1.6 Scope of the contribution

We do not yet establish novelty. We have not performed a systematic comparison against prior work in artifact evaluation or computational reproducibility assessment, and the four-layer structure proposed here has not been compared against existing taxonomies of validation failure. A reviewer should treat the contribution as a reproducible evaluation instrument and a primary dataset, not as a demonstrated advance over prior methodological work.

That paragraph is not a formality, and it belongs here rather than only in the limitations, because a marker in a limitations section is easy to read past. Section 8.7 carries the same statement as a standing limitation and keeps the marker for the work that is outstanding. We would rather a reviewer reach for the reference list and find it empty than find it full of plausible citations to work we did not search, because that second outcome is this paper's own third failure class — semantic relation failure, in which a structure is named after something it does not contain — applied to the manuscript itself.

## 2 Conceptual Model

The model has four units. They are not four names for one thing, and the central result of this paper depends on their being kept apart.

### 2.1 U1 — Claim

A claim is a proposition in the manuscript that is falsifiable against some artifact. It has exactly one of three statuses, and the three are not interchangeable. **CLAIM-CORRECT**: an artifact contains the asserted value. **CLAIM-CONTRADICTED**: an artifact contains a *different* value, so the falsification is measurable. **CLAIM-UNSUPPORTED**: no artifact bears on the claim, which is therefore neither confirmed nor refuted.

The third status is the one that carries the most methodological weight. A claim with no artifact and a claim with a contradicting artifact occupy different epistemic positions, and merging them inflates the appearance of evidence in a way that is invisible in the aggregate. An earlier version of our rubric recorded both in a single column; that is the root of the aggregation defect documented in Section 4.4.

## 2.2 U2 — Artifact

An artifact *exists* if a committed file contains the result. It *reproduces* if an auditor regenerated it and recorded the output, the exit code and the environment. A verifier that runs and disagrees is neither absent nor correct: it is an artifact that actively misinforms, and it is scored as contradicted rather than as partial support. The reproduction environment is recorded per execution, because a committed JSON that does not reproduce across toolchain versions is a finding about the artifact’s version scope and not a nitpick.

## 2.3 U3 — Relation

The relation asks whether the artifact actually evidences the claim. This is the unit the earlier layer rubric lacked, and it is where the consequential failures live. Two artifacts can both exist and both run, and the relation between one of them and its claim can be entirely absent. Four packages in our corpus exhibit it, and two of the four hold the strongest reproduction results in their own batches: one reproduces its artifact bit-for-bit, and one reproduces it byte-for-byte on every scientific field with a single wall-clock field differing. The four mechanisms are a thirty-two-state exhaustive enumeration described as a variational ansatz, with its mixer Hamiltonian and its four variational angles each constructed and never read; a pair of noisy-basis quantum parameters described as a variational optimiser, executed as a uniform random search; a constellation model that does not build the geometry its headline finding is a consequence of; and a set of three published detector names cited in the bibliography and replaced in code by three weighted clipped z-scores. The remaining two reproduce their artifacts with differences of one or two units in the last place, so the class is not an artefact of imperfect reproduction either — it is present at every level of reproduction fidelity in the corpus.

Relation failures are not arithmetic errors. They survive reproduction and they survive a number check, and they are the failure mode a conventional reproducibility benchmark cannot detect by construction.

The unit applies to the evaluation apparatus as well as to the artifacts under audit, and our own dataset supplies the demonstration. The aggregate summary of this corpus reconciled perfectly against its batch reports — every count matched, every roll-up reproduced — while asserting that a layer named L6 measured internal consistency, which the rubric and all three primary records place at L7. The same shift had propagated, by transcription, into the figure script for this manuscript and into the axis label of Figure 2, which has since been corrected. Every count involved was correct throughout. Section 4.4 documents it in full. We name the scope here because it is the scope that matters for method: a relation verdict is owed by every layer that asserts what another layer means, and a summary layer is such a layer. An evaluation instrument that adjudicates correspondence in thirteen packages and not in its own reporting is applying its rubric selectively.

## 2.4 U4 — Package

A package is scored on the layer stack of the rubric, and is **SUPPORTED** only if every layer holds. Any contradicted claim moves a package off **SUPPORTED**, and the verdict also accounts for whether the *central* claim is contradicted rather than merely whether some peripheral number is wrong. That distinction is deliberate and consequential: a package whose detector ranking is wrong in a peripheral section is a different object from one whose headline result is wrong, and a rubric that cannot express the difference cannot measure it.

## 2.5 Why the layers are non-equivalent

The four units are non-equivalent in a specific, checkable way: the sets they select are not nested, and their sizes do not move together. The dataset of record holds thirteen rows. Thirteen of thirteen artifacts exist, ten of thirteen reproduce, and zero of thirteen packages are fully supported. The relation between an artifact and a claim is broken by contradiction in all thirteen rows and is entirely absent in eight. No layer is a function of the layer beneath it, and in particular reproduction is not a proxy for integrity: of the ten rows whose artifact reproduces, seven are **PARTIALLY SUPPORTED** and three are **CONTRADICTED**; of the three that do not reproduce, one is **PARTIALLY SUPPORTED** and two are **CONTRADICTED**. The **SUPPORTED** column is empty in both blocks.

The practical consequence is that a single headline number is always a compression of several distinct questions, and a reader cannot recover them. The remainder of this paper reports them separately and always with their denominators.

# 3 Method

## 3.1 Corpus construction

The corpus is a set of thirteen scored rows over twelve distinct packages. Ten packages form the primary set — **adan**, **camae**, **isac**, **sc**, **ems**, **qce**, **quantum-k-sat**, **sra**, **icft**, **mega-constellation** — and two previously audited packages, **h266** and **cd**, were re-scored under the common rubric to establish a fixed denominator instead of two incomparable ones. The **cd** programme contributes two scored rows rather than one, because it contains two artifacts: the upstream manuscript, scored against the

upstream artifact, and the later audit manuscript written about it, scored against its own committed data. Same research programme, two artifacts, two verdicts; the earlier rubric scored them as one and the correction is recorded in Section 4.4.

The corpus is a convenience set — the published packages of one author that were auditable — rather than a probability sample of a defined population. We do not claim it is a sample of anything, and per rule R5 of the rubric it characterises nothing outside itself. Section 8.1 states what the author control costs, and Section 8.2 states why the size of the set licenses no inference at all.

### 3.2 Frozen rubric v1.1

The rubric is versioned, and the version used for adjudication was frozen before adjudication began. v1.0 defined eight layers — artifact, reproduction, numerical claim, figure, method, selection, internal consistency, verdict — and produced a correct gradient, but it left the unit of analysis implicit. That produced a real error: an early draft of the aggregate summary conflated “the package verdict is contradicted” with “the package contains at least one claim contradicted by its own artifacts”. Both are true. They are different metrics with different denominators, and conflating them in a paper about metric conflation would have been a defect in the paper’s own integrity metric. v1.1 fixes the four units, fixes the three claim statuses, fixes the package-level adjudication rules, and records the *cd* split.

The rubric also carries eight standing rules, of which three shaped the results. R1, artifact before claim: never read the paper’s number first; read the artifact, record its value, then find the claim. Inheriting a prior finding without checking is the failure mode the process exists to prevent, and Section 4.4 records an instance in which an evaluator declined to do exactly that. R5 and R6, no extrapolation and preserve the denominator: from  $N$  audited packages the report covers exactly those  $N$ , and a percentage without its denominator is not a result. R7, absence of contradiction is not evidence of presence: what survives must be recorded as carefully as what fails, which is the subject of Section 9.4.

### 3.3 Claim adjudication

For each package the manuscript was enumerated for falsifiable propositions. Each was traced to an artifact path and adjudicated to one of the three statuses of Section 2.1. Every contradicted entry records *both* values — claimed and actual — and the artifact path, so that the adjudication can be re-checked against the file rather than against this paper’s prose. Tables, multi-cell figures and mechanism identities are enumerated at whatever granularity the rubric’s U1 definition specifies, and that granularity is the single largest limitation of the claim-level results (Section 4.3).

A claim with no committed code path that could produce it is **UNSUPPORTED**, not **CONTRADICTED**, and the distinction is not cosmetic: in one package the entire ablation table has no code path that can produce it, and the honest report is that the table is unverifiable from the package, not that it is false.

### 3.4 Artifact reproduction

Every package was cloned read-only into a disposable directory and executed. The recorded protocol is: run the simulator unmodified and record the exit code and wall time; compare the regenerated artifact byte-for-byte with the committed one; where a difference appears, establish whether it is material before recording it as a finding. Non-bit-identical reproduction is not by itself a discrepancy. One package’s artifact matched on 98 of 102 numbers with the four exceptions differing by one or two units in the last place, from BLAS reduction order, in a field the paper reports to two decimals; that is a reproduction note, not a contradiction, and recording it as a contradiction would have been a false finding.

Where a committed artifact did not regenerate, the divergence was traced to its source before adjudication. In two packages the cause is a toolchain-scope effect: the artifact is valid for one library release and for no other, which is a genuine version-scope finding about the package and not a property of the method. In one of the two the cause could not be pinned to a specific release, and the reproduction failure is reported as measured with its cause unestablished.

### 3.5 Relation analysis

Relation analysis is where the earlier rubric had nothing to record, so the procedure is stated explicitly. For each claim with a named mechanism, the evaluator establishes what the code actually does, and records the answer as one of three outcomes. *Correspondence*: the implemented mechanism is the described mechanism. *Absence*: the structure named by the claim is constructed, or would have to be constructed, and is not present or is never used; the described mechanism is not implemented at all. *Contradiction*: the mechanism is implemented and the result it produces negates the claim.

The three outcomes are not scored against the code’s aesthetics but against the claim’s text. A 32-state exhaustive enumeration is not a defect as an algorithm; it is a correspondence failure when the manuscript describes a variational ansatz, because the reader’s inference from the text is then unsupported. Establishing correspondence empirically rather than by reading is possible and was done: one published policy table was confirmed by executing the committed optimiser at all four of its risk levels and comparing the returned masks, not by reading the source.

### 3.6 Package-level adjudication

The package verdict is assigned from the layer stack, not from the claim counts. **SUPPORTED** requires every layer to hold. **PARTIALLY SUPPORTED** requires real reproducible artifacts with a bounded set of contradicted claims. **CONTRADICTED** is assigned when the paper’s central claim is contradicted by its own artifacts. **UNVERIFIABLE** is assigned when the audit could not establish enough to judge, and an unverifiable layer is reported as a result rather than omitted. No package in the corpus was **UNVERIFIABLE**; every one of them ran.

One verdict was contested and is worth reporting because the rubric, not the package, was the problem. The earlier rubric’s reference case was scored as a single **PARTIALLY SUPPORTED** package. Applying the rubric’s own verdict text mechanically to the upstream manuscript yields **CONTRADICTED**, because the contradicted claims include the paper’s two stated conclusions; the **PARTIALLY SUPPORTED** verdict belongs to the audit manuscript written about that package. The rubric was corrected to score the two artifacts separately, and both readings are retained in the dataset so that the choice is visible rather than absorbed.

### 3.7 Bias and author-conflict controls

The corpus was authored primarily by the first author, who therefore had direct knowledge of the implementation history and publication context of every package. To reduce confirmation bias, the evaluation criteria, the claim unit, the verdict vocabulary, and the package-level adjudication rules were fixed and frozen before adjudication. Negative findings were retained without exception, and every self-correction made by an evaluator during adjudication is retained in the audit record.

That declaration is a description of an experimental property, not an editorial confidence exercise, and the framing matters for what the paper can claim. An author-controlled corpus is the only way to separate a badly designed artifact from one designed to mislead. It is also the reason the corpus cannot support a prevalence claim, and we do not make one. The weakness and the strength are the same fact, and Section 8.1 treats the cost.

Three further controls are recorded as part of the method rather than as appendix material. First, artifact identity was established before any claim was read, by hash, and the hash comparison in one package is three-way: the regenerated artifact, the committed upstream artifact and the audit paper’s committed copy agree byte-for-byte. Second, the duplication check was run over every committed artifact in every package, not only over packages, and the only duplication found was intra-package, with the same figure shipped in two or three byte-identical copies. Third, a suspected defect class was checked explicitly in every package and its *absence* was recorded where the absence held, because a hazard checklist that only produces hits is not a check.

### 3.8 Three layers of instrumentation, and what each can do

This study uses three kinds of instrument, and the temptation to treat them as one — or to quote the strength of the strongest for the whole apparatus — is the error Section 6.4 documents in our own. They are a hierarchy because each answers a different question, and none of them is a substitute for another.

**Regression guard**  
detects known failure modes.

**Independent derivation**  
tests quantitative correspondence.

**Clean-clone falsification**  
tests whether the published package survives adversarial reproduction.

*Regression guard.* The instrument of this kind in this repository is `verify_dataset.py`. It detects strings and shapes previously recorded as defects: its stale-figure patterns are parsed out of the dataset’s own correction log at run time, so logging a correction arms a guard without editing the script, and its layer-label and cross-batch-span assertions encode two specific defects found by hand. We say plainly what this is and is not. It is a regression guard, not a discovery mechanism. It can find a defect that has been written down somewhere; it cannot find one that has not. The falsification of Section 7.1 is the demonstration, not a hypothetical: the mispaired rate of Section 4.4 was invisible to the instrument set of the time because no string in the script could have contained a value it had never been told about. A guard built only from the defect record has no coverage over the part of the space the record does not describe, and that part is the part an integrity instrument is bought for.

*Independent derivation.* This is the batch reports, and the audit in Section 5.6. Its question is quantitative: does this number, recomputed by a path that does not go through the author’s transcription, agree? It is the only layer of the three that established the correspondence of the thirteen-row table to the three primary records, and it is the layer that isolated three discrepancies from thirty-five quantities in Section 5.6. It cannot establish correspondence, which is a question about meaning rather than about value.

*Clean-clone falsification.* The question is whether the published package, as a third party receives it, survives an adversarial attempt to break it. This is the only layer that examined the manuscript rather than the data, and the only layer that found

the pipeline's provenance guarantee to be unimplemented. It is also the only one whose findings were not available to us at the time we wrote the pipeline, which is its entire methodological point and is developed in Sections 7.1 and 6.4.

The three are not ranked, and we resist any phrasing that would rank them. The guard is cheapest and covers the least; the falsification is most expensive and covers the most; the derivation sits between them and is the only one that produces the corpus. A finding of this paper is only as strong as the weakest instrument that could have missed it, and the useful question about any single check is always which of the three layers it belongs to.

### 3.9 What the generated figures prove, and what they do not

Both figures in this paper are produced by a committed script that recomputes every derived value it draws and aborts with a non-zero exit, writing neither file, if any recomputed value disagrees with the frozen record. The gradient in Figure 2 and every row of Table 3 are therefore not hand-transcribed: if the transcription and the frozen dataset diverged, the build would fail rather than print a number.

That guarantee has a definite shape, and stating its boundary is part of the method rather than an apology for it. The script demonstrates that the *presentation layer cannot silently diverge from the frozen dataset*. It does not demonstrate that the frozen dataset is scientifically correct, that the simulations underlying the thirteen packages are valid models of what their manuscripts claim to simulate, or that the claims it adjudicates were the claims a reader would most want adjudicated. Those are judgements made by the rubric and by the evaluator, and they are exactly the judgements a script cannot audit. The chain of provenance from the row data to the printed figure is verified; the chain from the printed figure to scientific truth is not, and is not claimed.

The guarantee is only as good as the dependency graph behind it, and this one was rebuilt during this series because its first version did not support the claim above. The script locates the frozen dataset on disk and *parses it*: the row table, the derived-metrics block, the gradient block, the cross-batch block, the class-name list and the correction log are all read out of the dataset file at build time, and every expected value is compared against what that parse produced. There is no expected value written as a literal in the script. The two committed CSVs are then diffed against the dataset's own table cell by cell, so a transcription error is caught against the source rather than against another column of the same file. Bit-for-bit reproduction is not a column of the row table, so its count is parsed from the dataset's own per-row block and every repository it names is asserted to be a scored row.

This is the difference between a script that verifies and a script that confirms. Corrupting the dataset in a throwaway copy — the claim total and the cross-batch rate — leaves the script exiting zero and writing byte-identical figures under the previous design, because the dataset was not in the dependency graph at all: the check compared a CSV against constants that the same author had transcribed from the same markdown. It now exits non-zero and writes nothing, and `make_figures.py --self-test` proves that on every build by mutating the dataset text five ways and requiring the checks to fail each time. That rebuild is not a maintenance note. It is the subject of the case study in Section 7.1, which is where we report what the instrument asserted, what the audit demonstrated, why the published claim was left standing rather than weakened, and who found it.

The boundary is narrower than the word *generated* invites a reader to assume, so we name it rather than leave it implied. The script does not read this manuscript. **Table 1** is typeset by hand from the same transcription, so the script's guarantee runs from the frozen dataset to the figures and does not cover the typeset table. What covers the typeset table is a weaker instrument and we name it as weaker: the cross-file scan in `verify_dataset.py` reads this manuscript on every run and rejects it if it carries a pre-correction figure or a layer label that disagrees with the rubric. That is a guard against the defects recorded in the dataset's own correction log and not a cell-by-cell diff, and a reviewer should read it as such. Section 6.4 states what that residual limit cost us, because it is the sharpest evidence this study has that the problem generalises.

### 3.10 Clean-clone falsification as a rule of practice

The falsification attempt reported in Section 7.1 was run against a clean public clone, and we now state that as a rule of practice rather than as an accident of how this one attempt happened, because the rule is what generalises and the accident does not.

*Why the audit object must be fixed.* A working tree is not a reliable object to audit. It can hold unpublished fixes that the published package does not contain, so an audit run against it tests a version no reader receives. It can hold generated files, build outputs, or a modified dataset that overwrote the committed one, so the tree differs from its own history in ways no status output summarises. It can hold uncommitted work, local dependencies, an installed package shadowing a vendored one, or credentials and caches that make a script's environment unlike any reader's. And it can simply be at a different commit from the object under evaluation, which is the ordinary case rather than an exceptional one, because a working tree moves every day and a publication does not. Any of these makes a passing check uninformative: the check may have passed because the artifact is sound, or because the auditor was looking at something the reader will never see. A clean clone of the published state removes the ambiguity, because there is exactly one thing the reader could have received and the auditor tested that thing.

**Table 1: The thirteen scored rows, verbatim from the frozen dataset. Claims are counted at each row’s own adjudicated granularity; see Section 4.3. “Rep.” is artifact reproduction under auditor execution; “Verdict” is the package-level verdict of Section 3.6.**

| Package                             | Claims     | Corr.      | Contr.    | Unsp.     | Rep.      | Verdict  |
|-------------------------------------|------------|------------|-----------|-----------|-----------|----------|
| adan-ieee-package                   | 24         | 17         | 3         | 4         | Y         | PART.    |
| camae-paper                         | 52         | 47         | 5         | 0         | Y         | PART.    |
| isac-jasc-ieee                      | 81         | 70         | 11        | 0         | Y         | CONTR.   |
| sc-ieee-package                     | 38         | 34         | 4         | 0         | N         | PART.    |
| ems-ieee-package                    | 18         | 12         | 3         | 3         | Y         | PART.    |
| qce-ieee-package                    | 18         | 11         | 5         | 2         | Y         | PART.    |
| quantum-k-sat-ieee-package          | 25         | 13         | 11        | 1         | Y         | PART.    |
| sra-ieee-package                    | 14         | 9          | 5         | 0         | Y         | PART.    |
| icft2026-package                    | 25         | 11         | 13        | 1         | N         | CONTR.   |
| mega-constellation-optimization     | 16         | 6          | 6         | 4         | N         | CONTR.   |
| h266-drl-iccpr2026-package          | 19         | 8          | 11        | 0         | Y         | CONTR.   |
| cd-ieee-package (upstream)          | 21         | 3          | 17        | 1         | Y         | CONTR.   |
| cd-covert-anomaly-detection (audit) | 24         | 22         | 1         | 1         | Y         | PART.    |
| <b>Total, 13 rows</b>               | <b>375</b> | <b>263</b> | <b>95</b> | <b>17</b> | <b>10</b> | <b>—</b> |

*The conditions under which a falsification is valid.* Four, and we state them because an attempt that satisfies three of them is easy to mistake for one that satisfies four. *Clean public clone:* the audit runs against the published commit, with no access to the author’s working tree. *No working-tree access:* the auditor does not consult unpublished material, so the audit cannot be contaminated by knowledge of a fix already in progress. *No imports from the artifact under test:* the auditor’s derivation code shares no module, constant, or helper with the package it checks, so a shared bug cannot be silently agreed with. *Independent re-derivation:* quantities are recomputed from the primary records by the auditor’s own path, not read out of the package’s own outputs, so agreement between the two is evidence rather than tautology.

The third condition is the one that is most often skipped and that changes what the result means. Running a paper’s own verifier is circular, and not merely in the weak sense that it is a check by the checked party: it is a check *pointed at the layer it came from*. A verifier whose expected values are transcribed from the same table the paper quotes will confirm a transcription error in that table, because the error is present in both sides of the comparison. This series already contains the demonstration, and it is not a hypothetical one: the figure pipeline of Section 3.9 compared two CSVs against constants that the same author had transcribed from the same markdown, and the audit of Section 7.1 established that the dataset was not in its dependency graph at all. No corruption of the dataset could have been detected by it, and none was.

*What the rule does not license.* It does not make a falsification sufficient. A clean-clone falsification reports what survived and what did not, and its negative findings are as sample-specific as any other audit of a thirteen-row corpus. It does not license a prevalence claim, and we make none. It is also not free of the authorship concern of Section 3.7: the auditor was not a third party to the field and the corpus is still author-selected, which is why the claim we draw from the attempt is a claim about *instruments* and not about the population of research packages.

## 4 Dataset

### 4.1 The thirteen packages

Table 1 is the frozen row table of the dataset of record. It is reproduced verbatim, and every aggregate in Section 5 is recomputed from the same row data by the committed figure script, which aborts on disagreement. Two further instruments guard the corpus itself and neither is discretionary: `verify_dataset.py` joins every cell of the frozen row table against the per-row claim columns of the three batch reports, which are primary records written by three separate evaluators, and re-derives the cross-batch split from the row table per span. Section 3.9 states exactly how far the figure script’s guarantee reaches, and Section 6.4 states what the limit of that guarantee cost.

### 4.2 Three evaluation batches

Adjudication ran in three batches, and the batch structure matters because it determines which figures are comparable. Batch 1 scored four packages. Batch 2 scored four. Batch 3 scored two new packages and re-scored two previously audited ones under the common rubric, the `cd` programme contributing two rows. Batch 1’s report was reconstructed from a completed evaluator return after its scratch directory was rotated, with findings verbatim and only the file lost; that provenance is recorded here because it bears on the independence of batch 1’s counts.

Each batch’s evaluator worked read-only, in disposable clones, with the write scope limited to the batch report. No audited repository was modified, committed or pushed at any point in the series. Environment details are recorded per batch because they bear on reproduction: the recorded environments are macOS with Python 3.14.3, `numpy` between 2.4.2 and 2.5.3, `scipy` 1.18.1, `matplotlib` 3.11.2, and Apple `clang` with AddressSanitizer. Every artifact in the corpus was committed under an older

toolchain than these, and the toolchain caveat is disclosed in the batch reports rather than hidden: no discrepancy reported anywhere in the series is of a magnitude that could be attributed to toolchain drift.

### 4.3 Claim granularity as a recorded limitation

The per-claim rates in rows 1–8 and rows 9–13 were produced at *different claim granularities*. Batch 3 split compound sentences into separate atomic claims, which yields 50 of 105 at 47.6% over batch 3's five scored rows against batches 1–2's 213 of 270 at 78.9% over rows 1–8. This is a difference in counting convention and not in package quality, and the batch 3 evaluator flagged it unprompted before it was reconciled. Reporting the change rather than suppressing it is the point; the alternative was a trend line that would have been an artefact of the splitter.

Batch 3's report carries a second denominator as well, 28 of 81 at 34.6%, in which the `cd` programme is counted once as its upstream manuscript rather than as two scored rows. It is a different denominator and not an alternative value for the same one, and the two are not interchangeable. An earlier version of this paper paired the four-package rate with the five-row span, which inflated the apparent granularity gap from 31.3 points to 44.3; Section 4.4 records that as the fourth defect in the aggregation layer. The error was in a sentence of the summary layer and in no row, and it was in the direction that makes the finding more dramatic.

The pooled figures of 375 adjudicated, 263 correct, 95 contradicted and 17 unsupported are therefore an aggregate across two conventions. We report them as descriptive, and we do not present them as a defect rate for this corpus or as comparable to any single-batch figure. The layer and package verdicts are categorical judgements that do not depend on how finely a paragraph was split, and those are the defensible results. This is why the abstract leads with the categorical gradient and not with the claim-level percentage.

### 4.4 Correction log and dataset freeze

A paper on integrity cannot ship a dataset whose own summary fails an arithmetic check, so the correction record is part of the dataset rather than an appendix to it. The dataset was frozen on 2026-09-29, before any drafting.

Seven defects have been found in the aggregation layer and evaluation apparatus of this study, across five entries of the dataset's correction log. Four were in the aggregation layer: three arithmetic, and one not. Three were in the verification apparatus, and were found by an independent falsification attempt rather than by any instrument we had. The division is the result rather than an incidental detail. The three arithmetic defects were all found by one procedure — reconcile the summary block against the thirteen per-package rows — and that procedure is now part of the method. The fourth aggregation defect was a semantic relation failure inside the evaluation apparatus: every count in the summary was correct, and every label attached to those counts was attached to the wrong layer. We present the arithmetic three first and briefly, the fourth at length, and the apparatus three last and at more length still, because the arithmetic ones constrain nothing and the apparatus ones changed what the method has to check. The same reconciliation finds the arithmetic defects every time and will find them again.

**The pre-freeze aggregate error.** The first draft of the derived-metrics block stated 315 claims adjudicated, 94 contradicted, and “83% of adjudicated claims hold”. Machine verification of the thirteen rows gives 375 adjudicated, 95 contradicted, and 70.1% correct. The error was in the summary block only: every row total in the table was correct, and each row summed correctly. It was caught by hand-check against the rows before any figure was cited anywhere, corrected in place, and the pre-correction values are used nowhere in this series as results. They are quoted in this paragraph and in the dataset's own correction log, because a correction record that does not state what was wrong is not a correction record; an earlier version of this sentence claimed they were quoted nowhere, which the sentence itself contradicted. The methodological point is not that a mistake was made. It is that the aggregation layer requires the same audit as the artifacts, and that a dataset which fails its own consistency check is the same failure class this paper describes, one level up.

**Two further arithmetic errors, found by this paper.** Re-running the row-level reconciliation for this manuscript surfaced two more figures in the frozen dataset's derived-metrics block that do not survive being summed back out of the row table. They are recorded in the dataset's own correction log, and Table 2 gives them. We state them here because the paper argues that the aggregation layer requires the same audit as the artifacts, and because suppressing a second occurrence of this paper's own subject matter would be the failure this paper describes.

In both cases the thirteen per-package rows were correct before the correction and are correct after it. Both failures were in the summary block, which is the same layer that failed once already, and the same procedure — sum the rows, compare the sum against the block — caught each of them. It caught the first by hand before the freeze and these two mechanically after it, which is why the two errors are of different kinds: a wrong summation and a miscount of a column that a summation would not have touched.

**The fourth defect, and the one the procedure could not have found.** The aggregate report for this corpus was corrected for the first time only after the manuscript's figures were derived, and the defect was not arithmetic at all. Its headline table labelled its rows L3 --- Figure, L4 --- Method, L5 --- Selection, L6 --- Internal consistency and L7 --- Verdict. Rubric v1.0 fixes those names as L4 --- Figure, L5 --- Method, L6 --- Selection, L7 --- Internal

**Figure 1: The two checks that can be applied to the aggregate, and the verdicts they returned on the same file. Reconciliation verifies the counts and passes. Correspondence verification verifies the names those counts are attached to and fails. The two checks are independent, and only one of them is a check a conventional reproducibility pipeline performs.**

```

batch reports
|
v
aggregate
|
+-- arithmetic consistency ----> PASS
|
+-- semantic correspondence --> FAIL

```

consistency and L8 --- Verdict, and all three batch reports score their rows under exactly that mapping. Every label from L3 upward was shifted by one and *every count was correct*: 13/13, 10/13, 6/13 figure, 3/13 method, 7/13 selection, 1/13 internal consistency, 0/13 verdict. Re-rolling the thirteen rows against the batch reports' own layer columns reproduces all seven counts exactly, so nothing in the arithmetic moved. The same file's prose contradicted its own table: it states elsewhere that one package is **SUPPORTED** at L4 on four figures with committed generators, which is L4 = Figure under the rubric and not what the table's L4 row said.

The shape of that figure is the whole argument. The aggregate sits between its primary records and the reader, and it was checked against its primary records in the one respect a reproduction check is built to verify — do the numbers add up — and it passed. What failed was the other direction: does the summary mean what it says it means. A row-level reconciliation can verify `sums = sums` perfectly while the labels stay shifted by one, because the counts are attached to a layer name and the layer name is the thing that was wrong. The counts were all correct. The meaning was not.

This is recorded in the dataset's correction log as a third entry, and the aggregate file was already public when the defect was found, so the correction is post-publication and cannot be made silently. Two further instances of the same class were corrected alongside it, both naming errors in the frozen dataset itself: its derived-metrics block labelled the internal-consistency line L6, and its cross-batch comparability section described the pooled claim figures as the pre-correction 315 / 263 / 94 / 17 rather than the corrected 375 / 263 / 95 / 17 appearing above it. The same shift had also propagated, by transcription, into this manuscript's own figure script, where it had reached the axis label printed in Figure 2. The label constant is corrected, the axis label now reads L7, and the figure has been regenerated. The counts were never in question at any point in that chain; only the name was.

The methodological consequence is sharper than the count of instances, and it is the reason this defect rather than the other three is the one this section leads with. The procedure that caught the first three defects compares sums against sums, and a label shift is invisible to it: it reconciles perfectly while naming the wrong layer. Detecting it required a different check — parse the layer labels in the aggregate and assert that the L1–L8 sequence matches the rubric's names in order — and that assertion is now part of this manuscript's verification script, where it is the check that fails if the shift is reintroduced. The stricter claim is therefore not that a summary must be arithmetically correct. It is that a summary must be checked against the instrument it claims to summarise, and a check that only verifies arithmetic verifies half of it.

Read through the units of Section 2, the fourth defect is a failure of the relation unit and of nothing else: the aggregate existed, was derived from records that reproduce, and its relation to the instrument whose measurements it reported was broken while its arithmetic held perfectly. Section 6.4 argues why we treat it as an instance of a class already in the taxonomy rather than as a class of its own.

**The fifth defect, a rate attached to the wrong denominator span.** This one is in the aggregation layer again, and it is the fourth time that layer has failed, so it is being precise about why it survived the procedure that caught the other three. The dataset's cross-batch section attributed 34.6% to batch 3 while naming rows 9–13 as batch 3's rows. Batch 3 contributes five rows, not four, because the `cd` programme contributes both its upstream row and its audit-manuscript row; the four-package figure of 34.6% counts `cd` once. Rows 9–13 give 50 of 105 at 47.6%. The correct rate for the span the section names is 47.6%, and the section now states both denominators and says which is which.

Row-level reconciliation passes on this. The rows sum correctly, the summary block agrees with them, and the rate itself is arithmetically right for the count beside it. What is wrong is the pairing of a rate with a span, and reconciliation compares sums against sums: it has no way to ask which rows a percentage is claimed over. The first three defects were caught by a human re-deriving the numbers by hand. This one was caught by an auditor who re-derived the cross-batch split from the row table and compared the re-derived rate against the rate the sentence attached to the span — and that check now runs on every invocation of `verify_dataset.py`, joined against the per-row claim columns of the three batch reports, which are primary records written by three different evaluators and are never transcribed by us.

**Table 2: The two arithmetic corrections, both caught by row-level reconciliation against the thirteen per-package rows. The row table was correct throughout; the summary block was not. Both are recorded in the dataset’s correction log. The log’s third entry — the layer-label shift — has no row in this table because it has no arithmetic content: every count in it was already correct and only the layer names were wrong, which is why row-level reconciliation reconciled it and passed. It is written up in Section 4.4 and drawn in Figure 1.**

| Quantity             | Recorded | Corrected |
|----------------------|----------|-----------|
| Verdicts (S / P / C) | 0/7/6    | 0/8/5     |
| Internal consistency | 2 of 13  | 1 of 13   |

Two features of this defect are worth recording because they bear on the method. The origin is traceable and the loss is not the batch-3 evaluator’s: that report states both denominators explicitly, twice, and names them “Denominator A” and “Denominator B”. The primary record was right and the error entered on the way in. And the error was not neutral in direction — pairing the four-package rate with the five-row span inflated the apparent granularity gap from 31.3 points to 44.3 — which is the same asymmetry the aggregate report records for the verdict roll-up, and it is why we state here that every one of the thirteen rows was correct before this correction and is correct after it, and that the granularity caveat of Section 4.3 is unchanged by it: 47.6% is still a rate conditioned by a different claim convention and is still not a defect rate.

**The sixth and seventh defects, in the verification apparatus.** Both were found by the same independent falsification attempt and neither is a number in the dataset. The sixth is that the figure pipeline’s provenance guarantee was not implemented: the script read two CSVs and compared them against constants transcribed by its own author from the same file, so corrupting the dataset left it exiting zero and writing byte-identical figures. The seventh is that the caption for Figure 2 described a panel the generator does not draw, which is this paper’s own third failure class occurring inside its own apparatus. Two further items found in the same pass — a derived column cross-checked against another column of the same file, and a disjointness assertion that cannot fail because one of the two sets is empty — are instrument defects rather than dataset defects and are described where they arise: the first in Section 3.9 and the second in Section 1.1. The taxonomy renames and the dead rubric citation that the same pass surfaced are described in Section 6 and in the dataset’s correction log.

The methodological consequence of the sixth and seventh is stated where it belongs, as a limitation of the method rather than as a defect in one artifact: Section 6.4 sets out why an apparatus whose checked source is a constant transcribed by its own author cannot be relied upon to detect a correspondence failure in itself, what the two structural fixes are, and which one we have made. The sixth defect is developed as a case of its own in Section 7.1, and what the audit established about the published quantities as a whole is reported in Section 5.6. The non-arithmetic defects of this section are one phenomenon rather than a list, and Section 4.5 states the synthesis.

The first is the verdict distribution, an arithmetic slip in the block’s hand-written roll-up. Batch 1 gives three **PARTIALLY** and one **CONTRADICTED**, batch 2 gives four **PARTIALLY**, and batch 3 gives one **PARTIALLY** and four **CONTRADICTED**: eight and five, not seven and six. All three batch reports agree with the rows on this point, and the error is in the block alone. The corrected figure is 0/8/5.

The second is internal consistency, and it is a miscount rather than a summation error, which is why the first round of reconciliation missed it. The block recorded 2 of 13. Only one row carries **SUPPORTED** in the recorded internal-consistency column: **sc-ieee-package**. The **cd** audit manuscript is **SUPPORTED** on every layer below its internal-consistency layer, and the earlier draft counted it on that basis, but its recorded verdict in the column being summed is **CONTRADICTED** — on a single sample-standard-deviation disclosure, where Table I states  $n - 1$ , the package’s own committed statistics file carries both estimators, and the printed values are the population ones. The column is the unit of record, and it is contradicted there. The corrected figure is 1 of 13.

Neither correction changes the central result, and both sharpen it. Zero of thirteen packages are **SUPPORTED**, ten of thirteen artifacts reproduce, and thirteen of thirteen packages contain at least one contradicted claim, all before and all after. The internal-consistency step of the gradient falls from a reported 2 of 13 to 1 of 13, so the gradient is steeper than the frozen block stated, not flatter. An error in the direction that makes a finding look weaker is still an error, and correcting it is what makes the steeper figure citable.

**Evaluator discipline.** The audit record contains eight evaluator self-corrections across the three batches. Three are methodologically material, and all three are corrections against the audit’s own prior conclusions rather than against the audited packages. An evaluator overturned the rubric’s own assertion that one package’s JSON artifacts were non-bit-reproducible, after eliminating a confound in the first determinism test: training two agents in a single process yields a false positive, because the evaluation routine samples from the policy, so the second evaluation inherits an advanced random-number stream. The clean test is two full process runs, and those are bit-identical. An evaluator declined to inherit five numbers — 69.27, 63.37, 56.71, 64.79, 61.23 — that appear in no committed artifact of the repository they were attributed to, and substituted its own

measurements instead. A third evaluator nearly published a false contradiction against the rubric’s own reference case through an off-by-one in a sweep index, caught it by executing the code rather than reading the artifact, and recorded the correction: the claim under test was correct and the index was wrong. Two further corrections are recorded in the batch reports, one of which withdrew an unsupported claim the evaluator had drafted about its own subject.

The rate is worth reporting for what it implies about the method rather than about the auditor. Corrections of this kind — an inherited number declined, a confounded probe discarded, a rubric premise overturned — are the observable signature of a procedure that was applied rather than performed.

#### 4.5 One phenomenon, observed five times

The seven defects of Section 4.4 are not seven findings, and presenting them as a list understates what the series shows. The three arithmetic ones are one species; the other four are a single phenomenon, and they are the ones the paper’s argument rests on. They appear below as five instances, because two of them are visible in more than one place — a published guarantee and the implementation that fails to deliver it are the same loss at two different points in the chain — and because the circularity of the self-check is a fifth instance the series documents in its own instrument rather than in the correction log. We state the two principles again here because this is where they are earned rather than asserted.

*An integrity check cannot establish correspondence if its inputs are themselves an unaudited transcription of the claims being checked.*

*Verification should be derived from evidence, not from a transcription of the claim under verification.*

Each of the five is a loss of one condition: *the semantic condition that makes a number, a label, or a check mean what the document says it means*. What varies across the five is only the carrier. A count is a carrier, a rate is a carrier, a published guarantee is a carrier, a check is a carrier, and an implementation is a carrier. In each case the value was present, the file was present, the build passed, and the thing that failed was the link between what the document says and what the artifact contains.

- (1) **The layer-label shift.** Every label from L3 upward in a headline table was off by one, and every count was correct. The row the summary labelled L6 --- **Internal consistency**, carrying 1 of 13, is L7 --- **Internal consistency** under the rubric and in all three batch reports. The number did not mean what the sentence said it meant, because the label attached to it named another layer. *Lost: which layer a count measured.*
- (2) **The de-scoped rate.** A rate of 34.6% was correct arithmetic over four packages and was attached to a span of five scored rows, which gives 47.6%. The percentage was right and the set it was claimed over was wrong. The direction was not neutral: it inflated the apparent granularity gap from 31.3 points to 44.3. *Lost: over how many rows the rate was computed.*
- (3) **The false capability claim.** This paper published the guarantee that the figure build aborts if the transcription and the frozen dataset diverge. The build did not do that, because the dataset was not in its dependency graph at all. The guarantee was a claim about what the instrument contained, and the instrument did not contain it. *Lost: what is in the dependency graph.*
- (4) **The circular self-check.** The structure behind the third is the one that made it: the self-check and the thing it checks shared an author and a timestamp. A block of constants transcribed from the dataset was compared against a CSV transcribed from the same dataset by the same transcriber. The check was real code and could fail, but what it verified was the consistency of a single person with himself, and it could not in principle observe a transcription error, because both sides of the comparison carried it. The disjointness assertion the same audit found is the same structure in its purest form: two disjoint sets, one of which is empty, are trivially disjoint, so the assertion is subsumed by an assertion already made. *Lost: that a check can fail for a reason other than the one it names.*
- (5) **The defective pipeline.** `figures/make_figures.py` read two CSVs and compared them to constants in its own source, so corrupting the dataset left it exiting zero and writing byte-identical figures. The implementation did not do what its own documentation said it did — which is the same loss as the third, one level down, and is why the two are one defect in two carriers rather than two defects. *Lost: that the code implements the guarantee the document attaches to it.*

The seventh defect of Section 4.4, the caption describing a panel the generator does not draw, is the same loss in a sixth carrier: the manuscript’s description of its own artifact. It is held separate above because it is the third of the three discrepancies the independent audit isolated (Section 5.6) rather than one of the five instances the correction log counts, and because it is this paper’s own third failure class occurring inside its own apparatus. Treating it as an instance of the same loss is what makes the series a finding rather than a list.

The contrast with the arithmetic three is what gives the claim its content. A wrong sum is caught by recomputing the sum: the number is wrong, but the number still means exactly what it says, and reconciliation against the rows finds it every time. In the five above there is no wrong number to recompute. There is a correct number, or a correct build, or a correct set of counts, attached to a description that does not survive inspection — and the description is the part no arithmetic check reads.

**Table 3: Derived metrics, each with its own denominator. No percentage appears without the count it is computed from. Claim-level figures are pooled across two claim granularities and are descriptive only.**

| Metric                                    | Count | Rate  |
|-------------------------------------------|-------|-------|
| <i>U1, claim level, 375 adjudicated</i>   |       |       |
| CLAIM-CORRECT                             | 263   | 70.1% |
| CLAIM-CONTRADICTED                        | 95    | 25.3% |
| CLAIM-UNSUPPORTED                         | 17    | 4.5%  |
| <i>U2, artifact, 13 rows</i>              |       |       |
| artifact exists                           | 13    | 13/13 |
| artifact reproduces                       | 10    | 10/13 |
| <i>U3, relation, 13 rows</i>              |       |       |
| relation broken by contradiction          | 13    | 13/13 |
| relation absent                           | 8     | 8/13  |
| <i>U4, package, 13 rows</i>               |       |       |
| internal consistency                      | 1     | 1/13  |
| verdict SUPPORTED                         | 0     | 0/13  |
| verdict PARTIALLY                         | 8     | 8/13  |
| verdict CONTRADICTED                      | 5     | 5/13  |
| <i>Package-level containment, 13 rows</i> |       |       |
| contains $\geq 1$ CONTRADICTED            | 13    | 13/13 |
| contains $\geq 1$ UNSUPPORTED             | 8     | 8/13  |

That is the sense in which this is the same failure one level up as the corpus it audits, and it is why the method of Section 3.8 distinguishes a guard that detects known defects from a derivation that can reach an unrecorded one.

## 5 Results

Table 3 gives the derived metrics, each with its own denominator. Every row of that table is recomputed from the same row data as Table 1 by `figures/make_figures.py`, which parses the frozen dataset at build time, compares each recomputed value against what that parse produced, and exits non-zero writing nothing on any disagreement. The gradient of Figure 2 is the same computation rendered.

### 5.1 U1, claim level

Of 375 adjudicated claims, 263 are correct, 95 are contradicted and 17 are unsupported. We do not report 70.1% as a defect-complement rate, for the reason given in Section 4.3: the 375 is a pool of two counting conventions, and the split is 213 of 270 at 78.9% for batches 1–2 over rows 1–8 against 50 of 105 at 47.6% for batch 3 over its five scored rows, rows 9–13. Batch 3’s four-package denominator is a different quantity, 28 of 81 at 34.6%, and is reported in Section 4.3 rather than here because it is not a split of the same pool. The direction of the split is a property of the splitter.

What the claim-level data does support is a distributional statement that is insensitive to the convention. The UNSUPPORTED category is 17 claims in total, spread over eight of thirteen rows, and those are the claims for which no committed artifact bears on the proposition at all: an ablation table with no code path that could produce it, four ablation numbers present in no code and no artifact, a two-word claim about correlation attached to a quantity the code defines as a single global scalar. A number that is transcribed correctly from an artifact that does not measure the claimed quantity is CORRECT at U1 and worthless at U3, and the UNSUPPORTED column is how the first phenomenon becomes visible.

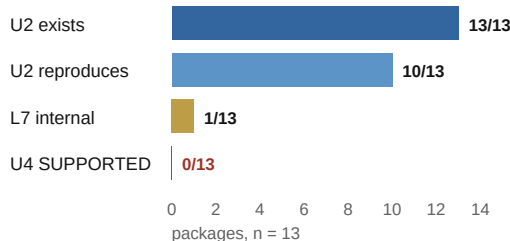
### 5.2 U2, artifact existence and reproduction

Every one of the thirteen rows ships a committed artifact: 13 of 13. Ten of thirteen reproduce under auditor execution, and among the reproducing group the reproduction is frequently exact. Bit-for-bit reproduction is a strictly stronger statement than *reproduces* — a committed JSON that regenerates across toolchain versions with the last digit of a float changed reproduces under the rubric — and it is not a column of the frozen row table. It was measured per package by each batch’s evaluator, it had no recorded field anywhere in this series until the correction pass that Section 4.4 describes added one, and an earlier version of this paper therefore quoted a count of it that no source in the corpus supported. The corrected figure, derived from the dataset’s per-row block and parsed by the figure script on every run, is **five of the thirteen scored rows**: the covert-detection upstream package and the audit manuscript’s copy of that same artifact, which agree with the regenerated file by a three-way SHA-256 — all three hashing to 4242d117dff67c716523e694704ca918f2537d4756cf1d4161938a629c237675,

## The artifact-to-claim gradient, and the denominators it depends on

13 scored rows over 12 distinct packages. Source: E1-DATASET-v1.0.md, parsed at build time by figures/make\_figures.py, which recomputes every quantity below from the committed row table and aborts without writing if any of them disagrees with the dataset.

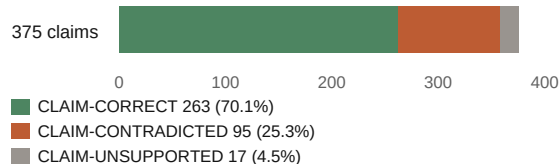
(a) Categorical layer verdicts,  $n = 13$



### Note: granularity conditioning

The 70.1% correct rate is computed over a pool of 375 claims that mixes two counting conventions. Batches 1-2, rows 1-8:  $213/270 = 78.9\%$ . Batch 3, rows 9-13:  $50/105 = 47.6\%$  over five scored rows;  $28/81 = 34.6\%$  over four packages, a different denominator in which `cd` is counted once. The split is a property of the splitter, so the pooled rate is descriptive and is not a defect rate for this corpus. Only panel (a) is comparable across batches.

(b) Adjudicated claim pool, 375 claims



### Note: verdict vs. containment

Package-level verdict: 0 SUPPORTED, 8 PARTIALLY, 5 CONTRADICTED, of 13. Packages containing at least one CLAIM-CONTRADICTED claim: 13 of 13. The second is the stronger statement and the more damaging one, and it holds for the package that is SUPPORTED on every layer below its verdict. Reporting only the first understates the finding; reporting the second as the verdict misdescribes the adjudication. The two sets are disjoint, but that follows from the SUPPORTED column being empty and is not an independent check.

### Note: aggregate reconciliation

2 arithmetic errors in the frozen dataset's derived-metrics block, found by row-level reconciliation against the 13 per-package rows and corrected with the rows left untouched. U4 verdict distribution: block recorded 0 / 7 / 6, corrected to 0 / 8 / 5. internal consistency: block recorded 2 of 13, corrected to 1 of 13. The correction makes the gradient steeper, not flatter: 1 of 13 rows are internally consistent. Neither correction changes the central result: 0 of 13 packages are SUPPORTED and 13 of 13 contain a contradicted claim before and after. Section 4.4 gives both rounds.

**Figure 2: The four-layer gradient and the denominators it depends on.** (a) The categorical layer verdicts, denominator 13 and constant across batches. Its third rung is labelled by rubric v1.0's layer name, `L7 internal`, because the frozen row table's column carries that name; the other three rungs are named by rubric v1.1's units, so the axis mixes two naming systems and the mixing is the dataset's, not ours. (b) The adjudicated claim pool, with the two-granularity conditioning stated in the figure rather than only in the caption. Beneath the two panels is a note row of three columns, not a third panel: granularity conditioning, verdict against containment, and the aggregate reconciliation. The last records two arithmetic errors found in the dataset's own aggregate layer by row-level reconciliation, both corrected with the row table left untouched; a third correction to the same layer — a layer-label shift carrying no arithmetic error at all — and two further corrections postdate this figure and are written up in Section 4.4. An earlier revision of this caption described a panel (c) that the generator does not draw, which is a correspondence failure of the third class inside our own apparatus; it is logged as such rather than quietly amended. Generated by figures/make\_figures.py, which parses E1-DATASET-v1.0.md at build time and diffs figures/data/e1.rows.csv against it cell by cell.

which is what makes it a three-way agreement rather than two files and an assertion; the reinforcement-learning package, byte-identical on every scientific field across two independent full process runs; the scheduling package, whose forty-five rows and five fields match exactly; and the co-design package, whose four policies and eight fields each match exactly. A sixth, the video-coding package, matches byte-for-byte on every scientific field with a single wall-clock field differing, which is a measurement of the machine rather than a result. Batch 1 contributes none of the five: the first package in it reproduces to

$2.2 \times 10^{-16}$ , the second to  $1.8 \times 10^{-12}$ , and the third reproduces 540 of 720 records bit-exactly while failing on precisely the 180 carrying its recommended operating point.

Three packages do not regenerate. Two of them diverge because the committed artifact is valid for one library release and for no other: in both cases the label count itself differs, so the random stream diverges inside the data generator rather than in the classifier, and the cause could not be pinned to a specific release. The third returns a different population size from the one its own committed results file holds, and that divergence is a code defect rather than version drift, because the documented contract of the routine is to return a fixed population and its peeling loop can terminate early.

The pattern that matters is not the count. It is that the three non-reproducing packages are no worse than the ten reproducing ones at the layers above, and Section 5.5 quantifies that.

### 5.3 U3, the artifact-to-claim relation

The relation is broken by contradiction in all thirteen rows, which is a statement about the corpus rather than a finding about any package, because every row also contains at least one claim that no artifact contradicts. The substantive U3 result is the second line: in eight of thirteen rows the relation between some artifact and some claim is *absent* — the structure the claim names is not implemented, is implemented and never used, or is replaced by something else under the same name.

The corpus contains three distinct absence mechanisms, and they are worth separating because they differ in how easily a reader detects them. In one, the named structure is constructed and discarded: the state vector, the mixer Hamiltonian and the variational parameters are each built and each never read, so the pipeline contains the vocabulary of a variational method and none of its mechanism. In one, the named structure is replaced rather than omitted: three published detector names are cited in the bibliography and appear as three weighted clipped z-scores, with no import of the corresponding libraries anywhere in the file. In one, the named structure is never built at all, and the artefact's real subject is a geometry that the manuscript's central empirical finding is a consequence of.

Absence of this kind is not detectable by any number check, because the numbers are consistent with the code that exists. It is detectable only by establishing what the code does against what the text says, which is why U3 is a separate unit and not a corollary of U1.

The frozen record stores the U3 status of each row as a binary, present or absent, and it does not store which of the three absence mechanisms applies to which row. We therefore report the count, 8 of 13, and the three mechanisms, and we do not assert a row-by-row mapping between them, because the dataset does not contain one.

**TODO(per-row U3 reason code): a frozen assignment of each relation-absent row to a specific absence mechanism.** The binary U3 column records that a relation is absent but not why, and the batch reports establish the three mechanisms without recording which rows they were observed in. Resolving this would require re-coding the eight rows against the U3 definition after the fact, which is re-adjudication and would break the freeze this paper reports against.

### 5.4 U4, package verdicts

No package is **SUPPORTED**. Eight are **PARTIALLY SUPPORTED** and five are **CONTRADICTED**. The dataset's derived-metrics block recorded seven and six until the row-level reconciliation described in Section 4.4 corrected it; the block and the row table now agree, and Table 2 records both figures.

The number that is easy to confuse with the verdict distribution is 13 of 13: every package in the corpus contains at least one claim contradicted by its own artifacts. That is the stronger statement and the more damning one, and it holds even for the single package that is **SUPPORTED** on every layer below its verdict — the *cd* audit manuscript, which holds 22 of 24 claims and whose one contradicted claim is a standard-deviation estimator disclosure. Reporting the verdict distribution without the 13 of 13 understates the finding. Reporting the 13 of 13 as though it were the verdict misdescribes the adjudication. Both appear, separately, in the dataset and in the note row of Figure 2.

A pre-freeze draft of the aggregate summary stated “12 of 13 packages have claims contradicted by their own artifacts”. That figure was wrong in both directions of its construction and is superseded.

### 5.5 Cross-layer disagreement

The most consequential result in the study is a joint one, and it is the one result in this paper that a machine check can fail. Of the ten rows whose artifact reproduces under auditor execution, *zero* are **SUPPORTED**: seven are **PARTIALLY SUPPORTED** and three are **CONTRADICTED**. Of the three rows that do not reproduce, one is **PARTIALLY SUPPORTED** and two are **CONTRADICTED**, and again none is **SUPPORTED**. Reproduction is not a usable proxy for integrity, and in this corpus it is not even weakly predictive of the verdict: the two groups have the same shape. `figures/make_figures.py` recomputes both blocks of that joint distribution on every run and aborts if either contains a single **SUPPORTED** verdict.

The sharpest instance is one package, and it is worth stating precisely because it is the case a reviewer will ask about. Its committed metrics file regenerates bit-for-bit across two independent process runs, on every scientific field, with one wall-clock field differing — the strongest reproduction result in the corpus. Its training loop executes a single policy update at the end

of a sixty-thousand-step rollout, where the manuscript describes per-episode updates over 120 minibatches, so the plotted convergence curve is the return trajectory of an untrained policy. Its own robustness file in the same repository reports the method at  $-10.26\%$  against its fixed baseline in stable conditions, with a probability of one of ever winning, and the manuscript cites neither that file nor any of the ten other robustness artifacts beside it. Fifteen of its mechanism identities are exactly right and both of its tables match its artifact in twenty of twenty cells.

The correct characterisation of that package is not that the code is broken. The trained policy differs measurably from a fresh initialisation, its action distribution flattens from a mode at  $34.1\%$  to a maximum at  $18.7\%$ , and it beats a uniform random policy  $67.93$  against  $62.90$ . What is false is the training procedure the paper describes, the convergence it reports, the significance evidence it offers, and the central advantage claimed in its abstract — and the refutation of that advantage is in the package’s own unreferenced file. A package can be perfectly reproducible and still be wrong, and reproduction tells you nothing about which.

## 5.6 What the independent re-derivation found

The clean-clone falsification of Section 3.10 re-derived thirty-five published quantities by its own path, without reading the paper, the committed CSVs, or the author’s scripts, and thirty-two of the thirty-five matched. That is the most valuable part of the audit and we lead with it for the reason it is easy to get backwards. An audit that reported only three discrepancies would be a defect-finder, and a defect-finder that finds three things tells a reader nothing about the other thirty-two. The thirty-two matching is what establishes that the chain holds everywhere the auditor expected it to hold, and it is what makes the three failures specific rather than ambient: they are three named breaks in an otherwise verified chain, not a general observation that the numbers are shaky.

Both derivation paths the auditor used — re-parsing the dataset’s own row table and derived-metrics block, and re-rolling the per-package layer rows from the three batch reports — agree with each other on every row-level and layer-level quantity. The three discrepancies are where the auditor’s results and the published artifact diverge, and they are conceptually distinct, which is what makes them useful instead of a count:

- **The de-scoped rate** of Section 4.4: a percentage attached to a span it was not computed over. A denominator and granularity failure — the arithmetic is right and the set is wrong.
- **A figure caption describing a panel the generator does not draw.** A failure of correspondence between a description and its representation: the caption names a third panel, the artifact contains two, and the content the caption assigns to the third exists only as an unlabelled note column.
- **An untraceable bit-for-bit count.** The paper asserted a count of rows reproducing byte-for-bit that appeared in no artifact in the corpus and in no field of the frozen record. A claim with no evidence behind it, which is the strictest form of the loss: there was nothing to check, and the number was not a transcription error but an assertion with no referent.

All three are U3. That is the point, and it is the reason the taxonomy of Section 6 is organised around the relation unit rather than around arithmetic. The three mechanisms are unrelated — one loses a denominator, one loses a representation, one loses the evidence itself — and no single check would have found more than one of them. A value-check finds none. A span check finds the first. A text-to-artifact correspondence check finds the second. Only an instrument that asks whether a number has any evidence behind it at all finds the third, and that is the one an author transcribing his own counts cannot supply.

*What this result is not.* Thirty-two of thirty-five is a result of one audit, on one corpus, under one protocol, and we do not convert it into a rate. It is not a reproducibility percentage, it does not estimate anything about packages outside these thirteen rows, and it must not be read as a score for the artifact. Its entire value lies in which three discrepancies it isolated and in the fact that it established the other thirty-two by a path that did not go through our own transcription. A headline metric would discard both and keep only the part that looks like a number.

## 6 Failure Taxonomy

The gap decomposes into three classes. We present them as a taxonomy of relations between computational evidence and scientific assertion, not as a list of bugs, because each class has a distinct signature in the units of Section 2 and a distinct detection requirement. Figure 3 renders the signatures. One further instance of this taxonomy occurred inside our own apparatus rather than inside a corpus package; we develop it in Section 6.4 and explain there why we classify it as an instance of the third class rather than as a fourth one.

### 6.1 Circular validation

The instrument reproduces the property it is meant to validate. The paradigm case in the corpus draws its estimator error from the very Cramér–Rao bound it is being tested against, and simultaneously holds the most reproducible artifact in the set, agreeing to  $1.8 \times 10^{-12}$ . It holds 55 of 55 cells of its summary table against its artifact, which is the cleanest table in the corpus, and its package verdict is **CONTRADICTED**.

### Three failure classes, read across the four units of analysis

Class names are the frozen dataset's own, parsed at build time from E1-DATASET-v1.0.md, section 'The three counterexample classes'; a rename in either the dataset or this script's input fails the build rather than shipping two vocabularies. Exemplars are transcribed from that section and from the cross-package findings in E1-AGGREGATE.md. A cell is a structural property of the class definition, not a count: it records whether the class is detectable at that unit at all.

| failure class and corpus exemplar                              | U1  | U2  | U3  | U4  | mechanism the method must be able to falsify                                        |
|----------------------------------------------------------------|-----|-----|-----|-----|-------------------------------------------------------------------------------------|
| <b>Circular validation</b><br>isac-jasc-ieee                   | NO  | YES | NO  | YES | Artifact reproduces a property derived from the assumption it purports to validate  |
| <b>Self-contradicting verification</b><br>ems-ieee-package     | YES | YES | NO  | YES | Verifier executes and its result contradicts the claim it is supplied to verify     |
| <b>Semantic relation failure</b><br>quantum-k-sat-ieee-package | NO  | YES | YES | YES | Artifact exists and runs but implements a different mechanism from the one it names |

Read the four cell columns down the three rows. Class 1 is invisible at U1, U2 and U3 at once: its artifact reproduces to 1.8e-12, the most reproducible artifact in the set, and no artifact contains a different value for the asserted proposition. It is detectable only at U4, and only because the assertion's own construction is the defect.

Classes 2 and 3 are both invisible at U2, because reproduction is not what is wrong with them: a verifier that runs and disagrees, and an artifact that runs and implements something else, both reproduce perfectly. The two failure classes a reproduction check cannot see are precisely the two the earlier unit set did not have.

Figure 3: The three failure classes read across the four units of analysis. A cell is a structural property of the class definition, not a count: it records whether the class is detectable at that unit at all. Classification and exemplars are transcribed from the frozen dataset's counterexample section and the cross-batch aggregate. Generated by figures/make\_figures.py from figures/data/failure\_classes.csv.

The mechanism is one line of code and it is not a bug. The simulator generates samples using a bound that the experiment then compares the estimator's error against. Agreement is guaranteed by construction, and the agreement is reported as a validation. Nothing downstream can detect this: the artifact exists, it reproduces, the number in the paper matches the number in the file, and the tables agree. The class is detectable only at U4, and only by an evaluator who asks what the simulation measures rather than whether it reproduces.

A second instance of the same class appears in the same corpus under a different name. A mitigation experiment defines two named techniques as scalar multipliers of a noise parameter — one as thirty percent less noise, one as fifty percent less noise — and then reports that the techniques improve performance. The figure demonstrates that reducing a parameter improves a result. It cannot demonstrate that the techniques reduce the parameter, because the reduction is the definition rather than the finding, and the paper's mechanistic conclusion, that measurement errors dominate gate errors, is drawn from the multiplier table and has no support in the code.

### 6.2 Self-contradicting verification

The artifact is executable and reproducible, and its result contradicts the claim it is supplied to verify. Three packages in the corpus ship a C reference kernel that compiles, runs, exits with status zero, and disagrees with the paper it is bundled with, and in all three the package's own README instructs the reader to compile and run it. In one the manuscript calls the file an independent reference kernel in its methods section and in its readme. The extension of this hazard beyond dead code is the finding: an unreferenced kernel is a tidiness problem, and a kernel that is advertised as verification and contradicts the paper is a correctness problem, because a reviewer who follows the instructions will run three programs that disagree with the papers, and in one case the disagreement reverses the headline ranking.

The mechanism is worth naming because it is not accidental. In that case the C implementation and the Python implementation are two different models: the C file gives one configuration a pass-through reflector while the Python sets every element weight

to zero, so no reflection at all occurs. The two codebases cannot serve as mutual checks, and the manuscript offers the C file as exactly that.

One package in the corpus carries a fourth variant of the same hazard in the opposite direction, and it is reported for balance. Its kernel is memory-unsafe and aborts on every run, writing 42 instrumented values into a 41-element buffer, which AddressSanitizer confirms as a heap overflow. That is a real defect, and the package does not present the kernel as a verification path: it ships a reduced-configuration test script and labels its own figures as generated by the simulator. The absence of the advertisement is what keeps it out of the class above, and it is recorded in the audit as a negative finding on a suspected defect.

### 6.3 Semantic relation failure

The artifact exists, can produce results, may even contain structures named after the claim, and the implemented mechanism does not correspond to the described one. Four packages in the corpus exhibit it, and the variety is instructive because the naming is intact in every case. The three class names used throughout this section are the frozen dataset’s own, and the figure script now parses them out of that file and fails the build if either side renames one; an earlier version of this paper published two of the three under different names from the dataset that is their source of record, which is this class applied to the taxonomy that names it.

A package titled as variational optimisation on a noisy basis is a full classical state-vector simulation in an array library: sixteen thousand complex amplitudes at the largest instance size, a parameter search that draws uniform samples and keeps the best, and a noise model that is a non-norm-preserving coherent admixture of a single basis state, renormalised away at read-out. Its reported score is the maximum over the ten most probable bitstrings read at full amplitude precision, presented as “measurement”, while the classical baseline it is compared against performs two thousand genuine random flips. The reported gap is therefore between an oracle and a real search, and the package’s own two reported quantities make the size of the oracle measurable: the reported satisfaction ratio corresponds to one violated clause, while the circuit’s expected number of violated clauses is more than five.

A second package asserts Pareto optimality in its abstract, its contributions, its results and its conclusion, and contains no Pareto computation anywhere in 696 lines. It contains an exhaustive enumeration of thirty-two states that returns exactly one point, minimising a single weighted scalar; a weighted-sum scalarisation produces a point on a front at best, and no front is ever computed. It also computes a state vector, a mixer Hamiltonian and four variational angles, and reads none of them; and its policy machinery executes to one constant mitigation mask and one constant performance level across all two hundred steps, so neither of the two components the abstract headlines as a conjunction influences the executed behaviour. The paper’s own tables report the two halves of that conjunction as interchangeable, and the abstract does not mention it.

A third package’s central empirical finding is that coverage improves as constellation size increases from ten to three hundred and twenty satellites, and its own results file shows the relation inverted, with coverage *falling* as size rises. The measurement that produces the inversion is a constellation model that does not build the pattern the paper names: twenty satellites per plane span  $21.375^\circ$  of arc at  $1.125^\circ$  spacing, so the whole three-hundred-and-twenty-satellite design occupies twenty distinct latitudes inside a band  $12.25^\circ$  wide. Four coverage percentages quoted in the results section do not occur anywhere in the artifact, where the maximum is below sixty per cent. The multi-objective optimiser itself is correct, deterministic across runs, and its geometry equations match the code exactly; the defect is in the constellation model, and the paper’s headline follows from it.

A fourth package describes a deep ensemble of three published detectors whose outputs are fused by learned weights. The three detectors are weighted clipped z-scores, with no import of any of the corresponding libraries present in the file; the fusion weights are a hardcoded constant that is never fitted; the wavelet feature block and the hidden Markov state sequence are both computed, logged and passed as arguments to a function that ignores them; and the reinforcement-learning agent, which is called five thousand times, has a monotone initial action-value table, so its greedy action is the ceiling threshold from the first step. Its realised action counts over those five thousand steps run from 1597 selections of the ceiling threshold to 61 selections of the lowest, and the two lowest thresholds together account for 128 of 5000. Two correlation coefficients quoted in its results section are attributed to a quantity the code defines as a single global average, so they are not weak, they are structurally incomputable. The package is nonetheless a well-built one: its committed results file is complete, its results table reproduces the artifact in twenty of twenty cells, its published negative result is stated correctly in its own results section, and it is the only package in the corpus to report that its proposed method underperforms its own baseline.

The naming is intact in every case in this class, and that is what makes the class coherent. A correspondence failure is not a missing mechanism; it is a present vocabulary attached to an absent or different referent. The defect in this study’s own aggregate layer has exactly that shape: every name was intact, every count was correct, and the referent each name attached to was the wrong layer. The next subsection treats it.

### 6.4 Why the aggregation defect is not a fourth class

The layer-label shift documented in Section 4.4 is the most consequential single defect this series produced, it is the cleanest available demonstration that aggregation integrity is not reducible to internal arithmetic consistency, and it is tempting to

promote it to a fourth entry in the taxonomy. We have not, and the reason is worth stating because the taxonomy is the paper's contribution and the temptation points at the paper's own thesis.

The aggregate asserted that a layer named L6 measured internal consistency when the rubric and the primary records call that layer L7. The counts were all correct; the meaning was not. That is the definition of the third class, applied at a different location in the chain: an artifact exists, is derived from records that reproduce, and implements a different referent from the one it names. The class is defined by a structural relation between a name and a referent, not by where in the chain the name lives, and on that definition the aggregate instance is a subtype of semantic correspondence failure rather than a sibling of it. Promoting a subtype to a sibling is the step at which a taxonomy becomes a list, and three reasons concrete to this study make the promotion actively wrong.

First, provenance. The three classes are transcribed from the frozen dataset of record, and each has an exemplar with a committed artifact behind it. The aggregate defect has no exemplar in that sense: it was found in the instrument, not in a corpus package, and asserting it as a peer class would require a definition the frozen record does not contain. Second, scope. This paper claims the three classes are exhaustive over the thirteen rows in which they were found, and Section 8.4 states that scope explicitly. The fourth instance was not found in the thirteen rows, so extending the taxonomy would silently widen the claim from a corpus finding to a general one — which is precisely the extrapolation rule R5 exists to forbid. Third, and decisively for a taxonomy whose stated value is that each class carries a distinct detection requirement: the label shift does not require a new kind of check. It requires the same check the third class already demands, applied one level up — establish what the code does against what the text says, and here, what the summary does against what the instrument defines. Two checks differing in where they are pointed is one method.

What the instance does earn is a place in the argument rather than in the taxonomy, and we give it one. It is the only case in this series in which correspondence failed inside the evaluation apparatus while arithmetic held perfectly, which makes it the sharpest available answer to the objection that a taxonomy of relation failures is a distinction without a difference. The apparatus is not exempt from the apparatus's own rubric.

*A limitation of the method, found by applying the method to itself.* The strongest form of this paper's thesis is not that summary layers can fail their own checks, but that *no self-check can be relied on to detect a failure in itself*. We did not have to argue that; we had to undergo it. The figure pipeline advertised that it aborts with a non-zero exit if any recomputed value disagrees with the frozen record, and Section 3.9 repeated the guarantee twice, in strong terms. It was false. The script read two CSVs, both transcribed by its own author from the dataset, and compared them against a block of constants transcribed by the same author from the same file in the same script. The dataset was not in the dependency graph at all. An independent falsification attempt corrupted the dataset in a throwaway copy — the claim total and the cross-batch rate — and the script exited zero and wrote figures byte-identical to the committed ones. A fifth defect, the rate attached to the wrong denominator span, was likewise invisible to every instrument we had; the blocklist of known-bad strings that guarded the typeset table could not contain a value it had never been told about. And a sixth, the caption for Figure 2 describing a panel the generator does not draw, was a semantic relation failure inside our own apparatus — this paper's own third failure class — that no instrument could have caught, because no instrument read the manuscript.

The pattern behind all three is structural rather than incidental, and it is the one thing we would ask a reviewer to take from this section. **The self-check and the thing it checks shared an author and a timestamp.** A block of constants transcribed from a markdown table, checked against a CSV transcribed from the same markdown table by the transcriber, verifies that the transcriber was consistent with himself. That is not nothing — it caught a real row-sum error during our own testing — but it is a far weaker instrument than the phrase “aborts if any recomputed value disagrees with the frozen record” implies, and it is weaker in a way no amount of additional care by the same author can fix. This paper's most emphasised methodological finding is that a summary layer can reconcile perfectly while mislabelling what its counts mean, and until the falsification attempt we had not applied that finding to our own pipeline. That omission is itself an instance of the class, and burying it would have been the failure class the paper is about. Section 4.5 states the general form of this pattern as the two principles of Section 1.2, and Section 7.1 develops what we did about it.

The fix is not a longer list of known-bad strings, and it is not a more careful review pass. It is one of two structural changes, and we have made the first. *Either* the checked source becomes a committed input rather than a constant, so that divergence between transcription and source is observable — which is what `figures/make_figures.py` now does, by parsing the dataset at build time, diffing the CSVs against it cell by cell, and proving on every run via `--self-test` that five distinct corruptions of the dataset each abort the build. *Or* the verifying instrument is authored independently of the artifact it checks, so that the two do not share an author and a timestamp — which is what found every one of the defects listed above, in a scratch clone, with derivation code that imports no module of ours and never opened our working tree. The first change is now in place for the figure pipeline. Neither is in place for the manuscript: `main.tex` is still typeset by hand, the guard over it is a correction-log-derived scan rather than a diff, and we do not claim otherwise. A dependency graph in which the manuscript is a committed input would close that gap; nothing in this series closes it today.

The honest generalisation is therefore a negative one, and we state it as a limitation of the method rather than a result about the corpus: *this study's own self-verification apparatus could not detect a correspondence failure in itself, and the failure it could not detect was in the same layer, of the same species, as the failures it was built to catch*. An integrity instrument whose coverage is reported only by its author should be read as covering less than its author believes, and the only available test of that coverage is an attempt to falsify it written by someone else. The findings of that attempt that bear on our own apparatus are precisely the ones Section 3.8 classifies as unreachable by a regression guard, and the rule for running the layer of the apparatus that can reach them — clean public clone, no working-tree access, no imports from the artifact under test, independent re-derivation — is stated as a rule of practice in Section 3.10 rather than left as a description of one audit.

## 7 Case Studies

The cases are chosen to make the taxonomy concrete and to answer the obvious objection that a corpus of negative findings. Each is drawn from the batch report that measured it.

The first case is not a corpus package. It is this paper's own figure pipeline, and it is placed here rather than in a methods footnote because it is the most instructive result in the study: it is a correspondence failure of the paper's own third class, inside the instrument built to detect exactly that class, found by an audit with no access to the working tree — and the response to it is the paper's methodological result rather than a repair.

### 7.1 The evidence chain: a claim left standing

*The instrument asserted an independence it did not have.* The figure pipeline for this paper claimed, in the manuscript and in its own documentation, that it recomputes every derived value it draws and aborts with a non-zero exit writing neither file if any recomputed value disagrees with the frozen record. The stated consequence was explicit: if the transcription and the frozen dataset diverged, the build would fail rather than print a number. That is a claim about the independence of the check, and the claim was false. The script opened two CSVs, both transcribed by its own author from the dataset, and compared them against a block of constants in its own source, transcribed by the same author from the same file. The dataset was not in the dependency graph at all. Independence is a property of where the expected value came from, and this expected value came from the same author's reading of the same document the check was supposed to police. This is the first principle of Section 1.2 stated as a property of code rather than of method: an integrity check whose inputs are an unaudited transcription of the claims being checked cannot establish correspondence, and cannot fail for the reason it advertises.

*The falsification demonstrated it with a corruption the build should have caught.* The independent attempt of Section 3.10 took a throwaway copy of the published package, corrupted the dataset's claim total and its cross-batch rate, and ran the build. The build exited zero and wrote figures byte-identical to the committed ones. That is the strongest possible form of the demonstration, and it is worth being precise about why it is stronger than a reported disagreement. A build that aborts on divergence is asserted by its documentation; a build that emits identical output from a corrupted source of record demonstrates, by construction, that the source of record was never an input. The instrument did not merely fail to catch the defect. It could not have caught it, for any value of the defect, because it was not looking at the file the defect was in.

*The claim was not weakened.* This is the decision the case turns on, and it is the one a reader should hold us to. The available responses to a published guarantee that the implementation does not support are to correct the claim, to correct the code, or to leave both standing and describe the gap. We took the second, and the reason is that the claim was the right description of the machine we should have been running. A build that aborts when the transcription and the dataset diverge is the correct guarantee for a figure pipeline. The defect was not in the assertion; it was in the dependency graph, which contained a block of constants where it should have contained the dataset. So the pipeline was rebuilt: `make_figures.py` now locates the frozen dataset, parses its row table, its derived-metrics block, its gradient block, its cross-batch block, its class-name list and its correction log, and derives every expected value from that parse. There is no recorded-value constant in the script. The two committed CSVs are diffed against the dataset's own table cell by cell, so a transcription error is caught against the source rather than against another column of the same file. And `--self-test` proves on every run that five distinct corruptions of the dataset each abort the build, by mutating the real dataset text in memory and requiring the real check to return failures — because a check that cannot fail is not a check, and this one is required to fail.

The published claim is therefore now true, and it is true because the implementation was raised to meet it rather than because the sentence was lowered to meet the implementation. A softened claim would have left the pipeline with the defect it has and the reader with a weaker promise; the defect was the thing worth removing. The claim was left standing, the chain was raised, and the build now aborts where it previously could not.

*The falsification was run by someone without access to the working tree.* The person who found the circularity was not the person who built the instrument, and the separation is not a detail of provenance — it is what makes the finding available at all. Every instrument in this series was authored by the same person as the corpus it checks. An author cannot independently derive the expectation for a check on his own transcription, because he wrote the transcription and knows what it was supposed

to say, and that knowledge is exactly what a falsification has to be able to do without. The auditor had the published clone, wrote their own derivation code, imported no module of ours, and never opened our working tree — so the unpublished fixes, the local environment, and the author's own recollection were all structurally unavailable as evidence. Under the rules of Section 3.10 this is what makes the result admissible: the audit object is fixed, and the auditor's expectation could not have been contaminated by knowledge of the defect.

*What the case shows.* It is the sharpest evidence in this study for the paper's own thesis, and it is sharper than any of the thirteen corpus cases because there is no alternative explanation available. The pipeline was not carelessly written, was not maintained by someone else, and did not fail on a machine unlike the author's. It was correct code doing exactly what it was written to do, and the thing it was written to do was not the thing the paper said it did. That is a semantic relation failure — Section 6.3 — in the paper's own apparatus, in the third class the paper names, and the paper's own instrument had no capacity to detect it. An evaluation instrument that adjudicates correspondence in thirteen packages and not in its own reporting is applying its rubric selectively, and the only available test of whether it is doing so is an attempt to falsify it written by someone else.

## 7.2 isac: circular validation with the best reproduction in the set

This is the paradigm case of Section 6.1 and it is worth the detail because it isolates the failure from every ordinary explanation. The package holds 81 adjudicated claims, 70 of them correct, 11 contradicted and none unsupported. Its artifact regenerates to  $1.8 \times 10^{-12}$ , the most exact agreement in the corpus, and 55 of 55 cells of its summary table agree with the artifact across five metrics by eleven signal-to-noise points. Its abstract's only headline number is refuted by the paper's own table. Its claim that the standard error exceeds four units across the range is contradicted by a sweep whose maximum is under three; its claim that the velocity spread stays below five metres per second is violated at nine of eleven points, reaching more than a hundred at the lowest signal level; and nine cells of its README table are wrong by up to a factor of thirty-three.

None of that is the interesting part. The interesting part is that the defect — estimator error drawn from the very bound it is tested against — is invisible to every measurement reported, and that the reproduction quality which makes the package look best-built in the corpus is orthogonal to it. Good simulation hygiene buys nothing when the defect lives in what the simulation measures.

## 7.3 ems: a verifier that reverses the conclusion

This is the paradigm case of Section 6.2. The package holds 18 adjudicated claims, 12 correct, 3 contradicted and 3 unsupported. All sixteen cells of its results table match the committed artifact to three significant figures, and the artifact regenerates bit-for-bit on every quantity the paper reports, on a toolchain far newer than the artifact's. Both headline values, and the security metric the abstract leads with, are correct. The Monte Carlo robustness claim — that no realisation falls below minus ten decibels — is measurably true, at zero of one hundred, with a minimum of  $-3.96$  dB, and it survived an auditor prediction that it would fail. Seven of its eight figures have committed generators that ran.

The package ships a 245-line C file that the methods section calls an independent reference kernel and the README instructs the reader to compile and run. It compiles, it runs, it exits zero, and it disagrees with the paper on three of four configurations. On one it differs by 11.55 dB with the *opposite sign*, which reverses the paper's central ranking: the kernel makes the pass-through configuration the best, where the paper's headline is that the steered configuration wins. The cause is that the two files implement different models, so the C file cannot serve as the check it is offered as.

The remaining defects are of the correspondence kind rather than the numerical kind, and they are worse for the paper's conclusion even though they change no number. The beam is steered at a hardcoded default angle that misdirects it by more than twenty degrees from the receiver, forfeiting nearly fifteen decibels of available gain. The "complete suppression" at the eavesdropper credited as the fundamental mechanism of the security improvement is a magnitude below the float64 cancellation floor for a sum of that many unit-magnitude terms, so the suppression is floating-point noise rather than physics. The stated shadow-fading standard deviation is twice the implemented one, and the surviving Monte Carlo robustness claim holds only because of that error: at the implemented value the probability of any realisation breaching the stated threshold is zero, and at the paper's stated value it is over half. One figure has no generator at all and its eight panels are identically zero, and it survives re-execution only because nothing writes it.

## 7.4 quantum-k-sat, qce, mega-constellation, icft

The four correspondence cases are developed in Section 6.3. Their claim splits are 25/13/11/1, 18/11/5/2, 16/6/6/4 and 25/11/13/1 for total, correct, contradicted and unsupported respectively. Two of the four hold the only clean figure layer and the best transcription fidelity in their batch, and one of the four is the corpus's most methodologically careful package on the layer that matters.

Three further points are worth recording because they bear on the taxonomy rather than on the individual packages. First, in one of them the significance test reported is impossible: the two arms are generated in separate loops and run on different

instances, so a paired test has nothing to pair, and the per-trial data needed to run any test is not committed at all. The effect size the paper reports is internally consistent with the paper’s own dispersion figures and externally inconsistent with the artifact, which yields a materially larger value. Second, in another, a headline percentage is arithmetically correct on an uncontrolled comparison: the three arms of the experiment each receive a different draw of the hardware noise parameters because the generator’s random stream is advanced between them and never reset, and the arm that wins received the best draw. Holding the hardware draw fixed across arms — an experiment the auditor wrote and ran — roughly halves the effect, and the effect survives. The right finding there is not that the result is fabricated but that it is the correct arithmetic on a comparison against a moving baseline. Third, in a fourth, the crossover operator does not enforce the declared decision-variable bounds while the mutation operator does, which is measurable over twenty thousand crossovers: 110 offspring fall outside the declared altitude range. The auditor’s first version of that test used two identical parents, which makes the operator a no-op and returned zero violations; the test was discarded and re-run, and the corrected test is the one reported.

## 7.5 cd as a near-positive case

This case answers the objection that a corpus of failures proves nothing but the selection of the corpus, and it is the control the method needs.

The `cd` programme contains two artifacts. The upstream package’s artifact regenerates byte-for-byte, confirmed by a three-way hash agreement between the regenerated file, the committed upstream file and a third committed copy — the strongest single reproduction result in the corpus. The audit manuscript written about that package holds 22 of 24 adjudicated claims, is supported on every layer below its internal-consistency layer, presents an unusually candid limitations section that flags its own novelty risk and declines to correct for a ceiling effect, and is the only manuscript in the corpus that is largely supported on its own evidence. Its central selection finding is the strongest in the corpus: one detector’s metric is pinned at its ceiling with exactly zero dispersion across all twenty seeds, so no significance test on that column can return a p-value, and the per-seed difference between the two leading detectors is exactly zero in seventeen of twenty trials, which makes the ranking the paper reports unrecoverable. It reports that, and it explicitly declines to select a harder operating point and re-run, on the stated grounds that doing so would change the experiment rather than audit it.

And it still contains a detectable artifact-to-claim contradiction. Its table declares that it reports the sample standard deviation on  $n - 1$ . Its own committed statistics file carries both estimators. The printed values are the population ones: for the autoencoder the population value rounds to the printed four decimals and the sample value does not, and the same holds for two of the other four detectors. The difference is under 0.001 on every cell and it changes no conclusion. It is a claim about method, it is falsifiable from the paper’s own artifact, and it is false.

That is the calibration the method needs. A five-figure package with a five-figure ledger, a bit-reproducible artifact, a hash agreement established three independent ways and twenty-two of twenty-four claims holding still carries a detectable artifact-to-claim contradiction. The defect is not a symptom of sloppiness at scale, and no threshold of build quality separates the packages that carry one from the packages that do not.

The upstream package is a different object and a different verdict. Its artifact also regenerates bit-for-bit, and its manuscript holds 3 correct, 17 contradicted and 1 unsupported claim of 21 adjudicated. Both of its stated conclusions are contradicted by its own artifact. Same research programme, two artifacts, two verdicts, and the earlier rubric’s single **PARTIALLY SUPPORTED** for the pair was the rubric’s defect.

## 8 Threats to Validity

### 8.1 Author-controlled corpus

The corpus was authored primarily by the first author, who had direct knowledge of the implementation history and publication context of every package. Every bias that threatens an author-controlled corpus applies here, and the controls of Section 3.7 reduce but do not remove them. Specifically: the choice of which packages to include was not random and was made by the author; the claim enumeration reflects the author’s reading of what each manuscript asserts; and an author who knows that a package contains a defect is more likely to find it, which inflates the contradiction rate relative to a blind audit even when the adjudication is frozen and the negative findings are retained without exception.

The same fact is what makes the mechanism results possible. Distinguishing an artifact that is badly designed from one designed to mislead requires knowledge of intent that a third-party auditor does not have. The weakness and the strength are the same fact, and we state both rather than choosing the flattering one. Nothing in this paper is a prevalence estimate, and no percentage here should be read as describing packages outside the audited rows.

### 8.2 Corpus size, and why no inference is licensed

The corpus is thirteen scored rows over twelve distinct packages, drawn from one author. The absence of any inferential claim here is not a statistical power shortfall awaiting more data, and we do not frame it as one. There is no power problem to remedy, because the corpus was not constructed as a probabilistic sample of any defined population. It is the set of thirteen

research packages that exist and that were selected because they were auditable. No population was defined, no sampling frame exists, and no randomisation occurred. An interval estimate, a significance test or an effect size would each presuppose a sampling model in which these thirteen rows are realisations of a random draw from a superpopulation, and this study has no such model and cannot acquire one by auditing more packages under the same design. A fourteenth row would be a fourteenth adjudicated case, not a fourteenth draw. The correct statement is that *no amount of additional data from this design would license this inference*.

Two distinct things follow, and conflating them is how a design limitation comes to be misreported as a data limitation. The first is that no inferential claim is made, because the design does not license one; that is a property of the design and is not contingent on how much data the design produces. The second is that the two headline figures have margin on grounds that have nothing to do with sampling. 13 of 13 and 0 of 13 are saturated counts: the first is a tally over the audited set and the second is the rubric's own verdict rule applied to every row, so no plausible reclassification moves either. The intermediate counts — ten of thirteen reproducing, one of thirteen internally consistent, eight of thirteen with an absent relation — are likewise exact tallies of the frozen rows. They are reported descriptively, they carry no sampling error, and they should not be read as estimates of anything. A reclassification moves them by construction rather than by sampling, which is a different objection and the one Section 4.4 answers.

That the intermediate counts are the fragile ones is not hypothetical here: they are exactly the counts the aggregation layer got wrong, on more than one occasion, and on a third occasion the defect lay in the names attached to those counts rather than in the counts themselves. This is why the correction log of Section 4.4 treats a reconciliation step as part of the method rather than as a final proofread, and why reconciling against the rows is not on its own a sufficient check on a summary layer.

### 8.3 Mixed claim granularity

The claim-level figures pool two counting conventions, as Section 4.3 states. The pooled 375 and the 70.1% derived from it are descriptive, and the split between batches — 78.9% over rows 1–8 against 47.6% over batch 3's five scored rows — is a property of the convention rather than of the packages. The categorical layer and package verdicts do not depend on granularity and are the results this paper rests on. A second corpus scored under a fixed granularity is required before any claim-level rate from this line of work should be compared with anything.

**TODO(single-granularity re-adjudication): the total claim count and the correct rate at one fixed claim granularity across all thirteen rows.** A single frozen granularity would require re-adjudicating all thirteen rows under the rubric's U1 definition and re-freezing the dataset, which is a new corpus version rather than a correction to this one, and which cannot be performed without breaking the freeze this paper reports against.

### 8.4 Domain heterogeneity

The thirteen packages span radio propagation, covert detection, deep reinforcement learning for video coding, electromagnetic eavesdropping immunity, combinatorial optimisation for satellite constellations, market manipulation detection, event-driven scheduling, and quantum-inspired combinatorial search. There is no shared benchmark, no shared metric and no shared task. The taxonomy of Section 6 is therefore a taxonomy of the artifact-to-claim *relation*, not of any scientific domain, and its three classes are claimed to be exhaustive only over the thirteen rows in which they were found.

### 8.5 Reproduction environment

Every reproduction was performed on a single machine with a single toolchain family, newer than the one the artifacts were committed under. That cuts both ways and both directions are reported. It makes the reproduction results conservative in one sense, because a newer library is more likely to break an old artifact than an older one would, and the two non-reproducing packages whose cause is a library-scope effect were found this way. It also means the ten reproducing rows are reproduced on one toolchain and not on several, so this paper cannot distinguish a genuinely platform-independent artifact from one that happens to survive one toolchain. The one package for which a wall-clock field differs is reported as a hardware difference rather than a result difference, and that call is a judgement.

### 8.6 Rubric dependence

Every number in this paper is a function of a rubric, and a different rubric would produce a different set. The corpus is small enough that the package-verdict column is not robust to the rubric's verdict text: one package's verdict changed during this series, from **PARTIALLY SUPPORTED** to **CONTRADICTED**, on a mechanical application of the same rubric's own criterion to a different layer of the same research programme, and a second package's layer 2 verdict was overturned by an evaluator who declined to accept the rubric's own premise. Both changes are recorded in the dataset rather than absorbed, and the rubric was corrected to score the two artifacts of that programme separately. A reader who rejects the rubric's verdict text should expect the intermediate counts of Table 3 to move. The two headline figures should not.

There is no inter-rater reliability statistic anywhere in this series. Each batch was adjudicated by a single evaluator, batch 1 by a delegated agent and batches 2 and 3 by separate agents, and no row was independently double-coded.

**TODO(inter-rater reliability): an agreement coefficient over the twelve units of the four-unit scheme.** A second independent adjudicator would have to re-score a sample of rows from the artifacts without access to the first adjudicator's findings, and the resulting statistic does not exist in the frozen record and cannot be reconstructed from it.

## 8.7 Novelty has not been assessed

This manuscript cites no prior work, and that is a deliberate choice rather than an oversight: inventing references to satisfy a citation requirement would be a worse defect than the absence, and the two earlier papers in this series made the same choice and stated the same risk. The consequence is real and we state it plainly. No novelty assessment against prior work in reproducibility methodology, artifact evaluation, or research software engineering has been performed. We do not know whether a four-unit claim/artifact/relation/package decomposition with a frozen verdict vocabulary has been proposed before, whether the three failure classes have been named, or whether the same result has been reported on a different corpus. A reviewer should treat the novelty of the contribution as open, and the authors accept that a reviewer who performs the search we did not may find prior art. Section 1.6 states this as a first-class limitation in the introduction rather than leaving it to this subsection alone.

**TODO(novelty assessment): a positioning review against published work on artifact evaluation and reproducibility measurement.** This requires a literature search, an inclusion criterion and a comparison table against prior taxonomies of artifact-to-claim failure. Neither this manuscript nor the two earlier papers in the series has a reference list, so no such comparison exists anywhere in the material available to us.

## 9 Discussion

### 9.1 Reproducibility versus integrity

The empirical claim of this paper is narrow and we state it as a claim about thirteen rows: reproducibility and artifact-to-claim integrity are independent properties in this corpus, and the second is the one a reader depends on. Ten of thirteen artifacts reproduce and zero of thirteen packages are fully supported, and the joint distribution shows no relationship between the two. The mechanism behind the independence is not mysterious. Reproduction tests whether the code does what the code does. Integrity tests whether the code does what the paper says. These coincide only when the paper describes the code accurately, and the frequency with which they coincide is an empirical question about writing, not about software.

The practical implication is that a reproducibility result should not be reported as an integrity result, and an integrity result should not be compressed into a reproduction statistic. The most common compression we found in practice — including in our own dataset's first draft — is to quote a claim-level percentage without the granularity that produced it, and to quote a package-level verdict without noting that a single contradicted claim is sufficient to move a package off **SUPPORTED**.

### 9.2 Verifiers as evidence

The second discussion point is about the semantics of a verification artifact. Three packages in the corpus ship a C file described as an independent reference implementation, presented as corroboration, which disagrees with the paper when run. The general point is that a reference implementation is not evidence unless it is derived independently of the implementation it checks and unless its agreement is established by running both. In all three packages the C file is derived from the same understanding of the model as the Python file, so agreement between them is not independent confirmation of anything, and disagreement between them is evidence that one of them does not implement the paper. Either way the advertisement is wrong, and in one case it is wrong in a way that reverses a headline result.

The correct handling is cheap. A reference implementation that is wired into the test suite, that fails the build when it disagrees, and that is regenerated by the same command as the main artifact, is worth more than a paragraph describing it. A reference implementation that a reader is invited to run by hand, and that nobody has run, is a liability with a README attached.

### 9.3 The danger of executable disagreement

A verifier that runs and disagrees is worse than no verifier, because it transfers false confidence. The reader's update is not "this package's claim is wrong"; it is "this package was checked". Where the check is a paragraph of prose describing an independent implementation, and the implementation is a file nobody compiles, the reader receives the confidence without the check, and the package's own language supplies the warrant.

We think this is the most transferable finding in the study, and it is transferable precisely because it is not about a domain. Any artifact that advertises a verification path has an obligation to establish that the verification passes. In the corpus, three packages ship a C kernel that their README instructs the reader to compile and run as a check, and all three disagree with their own paper when run. One of the three does agree with its paper on one published table — the optimiser's policy table,

which the kernel reproduces exactly — while reporting a perfect reliability score against the paper’s headline figure. Partial agreement from an unexecuted verifier is the worst case, because it licenses exactly the inference the verification was supposed to prevent.

## 9.4 Implications for artifact evaluation

Three implications follow for anyone building or applying an artifact evaluation instrument.

First, fix the unit of analysis before adjudication begins, in the rubric rather than in the evaluator’s head. Our own corpus demonstrates the cost of leaving it implicit: the first pooled figure was wrong by a factor of 1.19 in the denominator and by 12.9 points in the numerator, and the two granularities are still pooled in the frozen record because the freeze forbids re-adjudication.

Second, separate the unsupported status from the contradicted status and never merge them in a single column. A claim with no artifact and a claim with a contradicting artifact are in different epistemic positions, and a merged column makes a package look better evidenced than it is.

Third, add a relation unit, and require at least one relation verdict per named mechanism. The three classes in Section 6 were all found by asking what the code does, not by asking whether the numbers agree, and two of the three are undetectable at every other unit. An evaluation instrument without a relation unit will pass all three.

A fourth follows from the defects in our own apparatus rather than from the corpus, and it concerns the instrument rather than the rubric. Distinguish what kind of check you are running, and do not report a guard’s coverage as an instrument’s coverage. The three layers of Section 3.8 answer three different questions, and the failure we documented is what happens when a regression guard — which detects the defects someone wrote down — is described in the documentation of a derivation, or of a falsification. Our own `verify_dataset.py` is a regression guard and we now say so plainly: it can find a defect that has been recorded somewhere, and it cannot find one that has not, and the mispaired rate of Section 4.4 was invisible to it for exactly that reason. It is still worth running, and the two assertions it gained from being pointed at the batch reports and at the dataset’s own block make it stronger than a blocklist. It is simply not a discovery mechanism, and an instrument whose strongest layer is a guard should say so before a reader infers otherwise.

What survived is the other half of the recommendation, and we state it as plainly as the failures. Section 9.4 records what held.

## What Survived

An audit that only finds faults is not a balanced audit, and the confirmed results are what make the negative ones defensible. Rule R7 of the rubric requires this section, and it is a required part of the method rather than a courtesy.

Three of these deserve comment because they cut against the paper’s own interest. The reinforcement-learning policy in one package genuinely works: it beats a uniform random policy by more than five units of its own quality metric, and its action distribution measurably flattens under training. The package’s failure is the training loop its manuscript describes and the robustness study it does not read, not the method. The multi-objective optimiser in another package is a correct, deterministic implementation, and four of its five system-model equations match the code to the digit; its failure is a constellation model that does not build the pattern the paper names. And a third package published its own negative result, stating in its results section that its proposed method achieves a lower score than its own baseline, and repeated it in its limitations. That is the opposite of the defect pattern the rest of the corpus exhibits, and it is why the package is **CONTRADICTED** rather than worthless: the author knew the headline was bad and published it anyway.

## 10 Conclusion

Correct numerical output is not sufficient evidence of artifact-to-claim integrity, and the separation is not a matter of degree. In a frozen corpus of thirteen scored rows, ten artifacts reproduce under auditor execution and zero packages are fully supported; every package in the corpus contains at least one claim contradicted by its own artifacts; and no reproducing row is **SUPPORTED**, which the committed figure script recomputes from the joint distribution on every run. The claim-level pool of 375 adjudicated claims, of which 263 hold, is reported as descriptive and is conditioned by two counting conventions, and it is not the result this paper rests on.

The gap decomposes into three classes that a reproduction check cannot detect and a number check cannot detect either: circular validation, in which the artifact reproduces a property derived from the assumption it purports to validate; self-contradicting verification, in which a supplied verifier runs and disagrees; and semantic relation failure, in which an artifact runs and implements a different mechanism from the one it names. Each was found by asking what the code does, not by asking whether the numbers agree, and each lives in the relation between an artifact and a claim, which is the unit an earlier layer rubric did not have.

The methodological results are the negative ones about our own instrument. The dataset’s aggregation layer failed its own consistency check four times, across four entries of the dataset’s correction log. The first three were arithmetic, and all three were found by one procedure: sum the thirteen per-package rows, then compare the sum against the summary block. The first

**Table 4: Recorded results that survived adjudication. These are the findings that make the negative results credible: each is a claim the audited package got right, confirmed by re-execution rather than by reading.**

| Package       | Surviving result                                                                                                                                                                                                       |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| isac          | 55 of 55 cells of the summary table agree with the artifact.                                                                                                                                                           |
| camae         | All 36 table cells and 45 of 45 summary cells exact.                                                                                                                                                                   |
| sc            | All 24 table cells and the adaptive-selection result hold.                                                                                                                                                             |
| ems           | All 16 cells of the main table; the Monte Carlo claim survived a prediction that it would fail, at 0 of 100 below $-10$ dB, minimum $-3.96$ dB.                                                                        |
| qce           | The published policy table was confirmed by <i>executing</i> the optimiser, not by reading it.                                                                                                                         |
| quantum-k-sat | All four figures have committed generators that ran; 13 of 14 transcribed numbers correct; no seed or best-case selection.                                                                                             |
| sra           | The results artifact regenerates bit-for-bit, 45 of 45 rows; zero unsupported claims; the makespan null result is disclosed in the manuscript.                                                                         |
| mega-const.   | The multi-objective optimiser is a correct deterministic implementation, two runs byte-identical; four of five system-model equations match to the digit.                                                              |
| h266          | Regenerates bit-for-bit across two fresh processes; fifteen mechanism identities exactly right; the trained policy beats a uniform random policy 67.93 against 62.90 and its action mode flattens from 34.1% to 18.7%. |
| icft          | Committed results file complete; results table reproduces 20 of 20 cells; its own negative result published in its results section.                                                                                    |
| cd audit      | Regenerates byte-for-bit by three-way hash; 22 of 24 claims hold; the strongest selection analysis in the corpus.                                                                                                      |

was caught before the freeze and corrected before any figure was cited. The second and third were found after the freeze, by the reconciliation this paper runs against its own data, and are logged rather than quietly amended: a verdict distribution recorded as 0/7/6 where the rows give 0/8/5, and an internal-consistency count recorded as 2 of 13 where the recorded column gives 1 of 13. In all three cases the thirteen rows were correct and the summary was not.

The fourth was not arithmetic, and it is the one that changed what the method has to check. The aggregate’s headline table carried its layer labels shifted by one from L3 upward against the frozen rubric and against all three batch reports, so that a correct count of 6 of 13 was attached to L3 — Figure where the rubric calls it L4. Every count reconciled; only the names moved. Because the aggregate file was already public, that correction is post-publication. It was found not by reconciliation but by parsing the label sequence and asserting it against the rubric, and that assertion now runs in this manuscript’s verification script. A fifth aggregation defect, a rate of 34.6% attached to batch 3’s five scored rows when that rate belongs to a four-package denominator and the five rows give 47.6%, survived every instrument we had: reconciliation passes on it, because a mispaired rate is not a wrong sum. Both the label shift and the mispaired rate are now caught by a check that joins the frozen row table against the three batch reports — primary records written by three separate evaluators, which we never transcribed — and that re-derives the cross-batch split per span from the row table. Eight evaluator self-corrections are in the record, including an evaluator who overturned the rubric’s own premise, an evaluator who declined to inherit five numbers that appear in no committed artifact, and an evaluator who caught an off-by-one that would have published a false contradiction against the rubric’s own reference case.

The aggregation layer requires the same audit as the artifacts, and a dataset that fails its own consistency check is the same failure class this paper describes, one level up. We report that failure four times over rather than once because two of the corrections move the reported gradient in the direction that weakens our own claim, and correcting them anyway is the only version of the gradient that is citable; because the fourth shows that the arithmetic check which caught the first three is not sufficient on its own; and because the fifth shows that even a check which is not arithmetic is not sufficient on its own. A summary layer must be checked against the instrument it summarises, and a rate must be checked against the span it is claimed over.

Read together, the four non-arithmetic defects of this series are not a list but one phenomenon observed five times, once per carrier, and it is the phenomenon this paper is about. Each is a loss of the semantic condition that makes a number, a label, or a check mean what the document says it means, and only the carrier differs: a count, a rate, a published guarantee, a check, an implementation. The general form is stated as two principles in Section 1.2 — *an integrity check cannot establish*

*correspondence if its inputs are themselves an unaudited transcription of the claims being checked, and verification should be derived from evidence, not from a transcription of the claim under verification* — and Section 4.5 develops both against the five instances. A reader who holds those two statements can classify a defect we have not seen, which is the test of whether a series of failures has produced a finding or a list.

Two further defects are in the verification apparatus rather than in the dataset, and they are the reason this paper now states a limitation of its own method rather than only of its corpus. The figure pipeline advertised a guarantee it did not implement: it compared a transcription against constants in its own source, never opening the dataset, so corrupting the dataset left it exiting zero and writing byte-identical figures. And a figure caption described a panel the generator does not draw, which is this paper's own third failure class inside its own apparatus. Both were found by an independent falsification attempt that wrote its own derivation code, imported no module of ours, and never opened our working tree. Section 6.4 draws the conclusion we think a reviewer should take from them: an apparatus whose checked source is a constant transcribed by its own author cannot be shown to detect a correspondence failure in itself, and the only available test of such an apparatus is an attempt to falsify it written by someone else. We have made one of the two structural fixes — the checked source is now a committed input, parsed at build time, and five distinct corruptions of it each abort the build on every run — and we do not claim the second, which would be a manuscript that is itself a committed input to a check authored by someone other than its author.

The response to that failure is the methodological result we would most ask a reviewer to weigh, and Section 7.1 develops it. The published claim was not weakened. The pipeline was rebuilt until the claim became true, because the claim described the machine that should have been running and the machine was what was wrong. The person who found the circularity was not the person who built the instrument, and that separation is the only reason the finding was available: an author cannot independently derive the expectation for a check on his own transcription. What the same audit established about the rest of the published chain is reported in Section 5.6 — thirty-five quantities re-derived by its own path, thirty-two matching, and three specific discrepancies isolated. Those three are a de-scoped rate, a caption describing a representation the generator does not produce, and a count with no evidence behind it; they are three unrelated mechanisms at one unit, and we report them as such rather than as a score for the artifact. Thirty-two of thirty-five is the result of one audit on one corpus under one protocol, and it is not convertible into a rate: the corpus is not a probability sample of a defined population, and the design licenses no such inference (Section 8.2). Its value is in which three discrepancies it found and in the fact that it established the other thirty-two by a path that did not go through our own transcription.

Four limitations are load-bearing and are stated as such. The corpus is author-controlled, which is a weakness for any prevalence claim and the only way to attribute mechanism. The corpus is thirteen rows and is not a probability sample of any defined population, so no interval estimate is offered — not because thirteen rows are too few to support one, but because the design licenses none and more rows of the same kind would not change that; the two saturated counts the paper rests on do not depend on the argument. The claim-level figures pool two granularities and are not comparable to any single-batch figure. And no novelty assessment against prior work in reproducibility methodology has been performed, because neither this manuscript nor the two earlier papers in the series has a reference list; a reviewer who searches where we did not may find prior art, and we accept that outcome rather than conceal the gap by inventing citations.

The immediate next step is a second corpus scored under a single frozen claim granularity, with independent double-coding on a sample, which is what would convert the categorical gradient reported here into a claim-level rate that could be compared with anything.